

LangChainV1.0重构

核心变化：更精简、更强大的 LangChain

LangChain v1.0是一个专注于构建代理的、可直接用于生产环境的基础框架，标志着 AI 智能体开发正式进入**工程化阶段**。此次更新最核心的变化是基于 LangGraph 重构了底层架构，并引入了全新的 Agent 构建 API。以下是 v1.0 的详细更新内容：



1. 核心架构变革：全面拥抱 LangGraph

LangChain v1.0 不再是独立的链式调用工具，而是成为了构建在 **LangGraph** 之上的“Agent 快速通道”。

- **统一运行时**：所有 v1.0 的 Agent 都在 LangGraph 运行时上执行。这意味着直接获得了 LangGraph 的高级特性：持久化、流式输出、以及“时光倒流”（Time Travel）调试能力。
- **定位差异**：LangGraph 负责底层的图编排和精细控制；LangChain v1.0 则提供开箱即用的高层抽象，几行代码就能跑起一个生产级 Agent。

2. 全新 API：create_agent

这是 v1.0 中最重要的 API，取代了过去的 AgentExecutor 和各种 Chain。

- **标准化入口**：create_agent 是构建 Agent 的新标准。它提供了一个比底层 LangGraph 更简单的接口，同时比旧版 Agent 更灵活。
- **替代旧版**：它直接替代了 langgraph.prebuilt.create_react_agent 以及旧版的 ReAct Agent 构建方式。

3. 引入中间件机制

这是 v1.0 最大的亮点之一，极大地增强了可扩展性。可以像在 Web 开发中一样，在 Agent 的生命周期中插入自定义逻辑：

- **拦截与控制**：你可以在模型调用前后、工具调用前后插入代码。
- **预置中间件**：官方提供了常用的中间件，例如：
 - **PII Redaction**：在发送给 LLM 之前自动去除敏感信息。
 - **Summarization**：当上下文过长时自动总结历史对话。
 - **Human-in-the-loop**：在执行敏感操作前暂停，等待人工批准。

4. 标准化内容块

解决了不同大模型（OpenAI, Anthropic, Google 等）之间消息格式不统一的痛点。

- **统一格式**：引入了 `content_blocks` 属性，提供了一种跨提供商的统一方式来访问文本、推理过程、引用和工具调用。
- **解耦**：将内容与原始字符串解耦，使多模态和复杂输出的处理更加类型安全。

5. 结构化输出改进

- **主循环集成**：结构化输出现在直接集成在 Agent 的主循环中生成，而不再需要额外的 LLM 调用。
- **降本增效**：这直接减少了 Token 消耗和延迟，模型可以选择直接返回结果或调用工具，无需复杂的 Prompt 工程技巧。

6. 命名空间清理与迁移

为了解决长期以来的“臃肿”问题，v1.0 进行了大刀阔斧的清理：

- **精简核心**：langchain 包现在只包含构建 Agent 所需的核心组件（Agent, Model, Tools）。
- **遗留代码迁移**：所有旧版的 Chains（链）、AgentExecutor 和遗留工具都被移动到了一个新的包 `langchain-classic` 中。
- **向后兼容**：虽然旧代码被移走，但通过安装 `langchain-classic`，仍然可以运行旧项目，同时逐步迁移到新架构。

7. 其他重要变动

- **Python 版本**：放弃了对 Python 3.9 的支持，建议使用 Python 3.10+。
- **LangGraph v1.0**：LangGraph 也同步发布了 v1.0 正式版，承诺核心 API 的稳定性。

总结：我该如何升级？

如果你正在开始新项目，直接使用 `create_agent` 和 v1.0 API。如果维护旧项目，建议先安装 `langchain-classic` 保持运行，然后逐步用新的 Middleware 和 `create_agent` 替换掉老的 Chain 和 Executor。

什么是LangChain？

LangChain是一个用于开发由大型语言模型 (LLMs) 驱动的应用程序的框架。

我们希望，大模型应用并不仅仅是聊天，更希望能从已有的数据库或文件中提取信息并根据这些信息执行具体操作。

LangChain 正是为了实现这一目标而诞生的。LangChain 是一个开源框架，它允许开发人员将像 通义千问、DeepSeek这样的大语言模型与外部的系统和数据源结合起来去完成更多更复杂的和操作行为。

当然，要实现这些功能，我们还可以借助Dify、RAGFlow之类的开源平台，但是自由度受限，对于很多企业复杂的业务适用度欠缺。我们也可以不用任何框架，全部自己完成，但费时费力。

LangChain已经帮我们准备好了很多组件，直接使用就行，但同时也提供了足够的自由度，让我们可以根据自己的情况做自定义的实现。

LangChain是一个基于大语言模型用于构建端到端语言模型应用的框架，它提供了一系列工具、套件和接口，让开发者使用语言模型来实现各种复杂的任务，如文本到图像的生成、文档问答、聊天机器人等。

官网地址：<https://python.langchain.com/docs/introduction/>

GitHub源码地址：<https://github.com/langchain-ai/langchain>

企业应用：摩根大通、梅奥诊所等机构已将其用于智能投顾、医疗诊断等场景。学术研究：斯坦福、MIT 等高校利用其设计模型对比实验和多模态融合方案



地位类比：

LangChain = Java 生态中Spring Boot 与 Apache Camel 的结合体

Spring Boot 式的 "脚手架"：快速构建应用。例如，通过 10 行代码实现智能客服系统

Apache Camel 式的 "集成总线"：支持 200 + 工具和服务，动态调用外部资源

LangChain = C++ 生态中的ROS + grpc

LangChain核心价值：重塑 AI 开发范式

通过标准化工具链和生态整合能力，将大语言模型（LLM）的原始能力转化为可落地的行业解决方案

标准化接口和开放生态所带来的行业普惠效应，对于企业而言，它是数字化转型的 "AI 操作系统"；对于大模型应用实时者，它是连接 LLM 能力与现实需求的 "数字桥梁"。

正如红杉资本在其 A 轮融资中评价："LangChain 正在定义下一代软件的开发范式。"

LangChain体系

LangChain体系概览



LangChain包含了好几个子系统，每个子系统各有各的用处和组成：

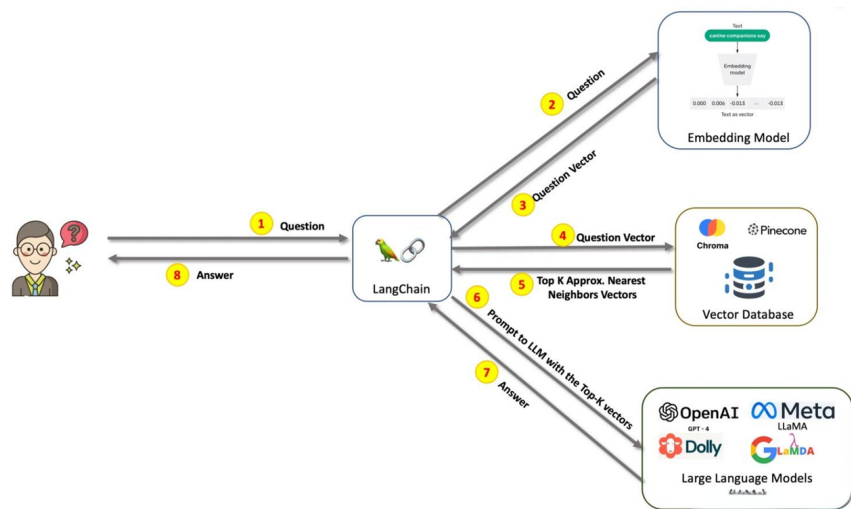
具体来说，该框架由以下开源库组成：

- langchain: 主要入口，包含构建 LLM 应用所需的所有实现。
- langchain-core: LangChain 生态系统中使用的核心接口和抽象。
- langchain-community: 第三方集成。
- langchain-classic: 旧版的 langchain 实现与组件。
- 合作伙伴库（例如 langchain-openai、langchain-anthropic 等）：一些集成已进一步拆分为自己的轻量级库，仅依赖于 langchain-core。
- LangGraph: 通过将步骤建模为图中的边和节点，构建强大且有状态的多参与者应用程序。与 LangChain无缝集成，但也可以单独使用。
- DeepAgents: 能够规划、使用子代理，并利用文件系统处理复杂任务的代理。
- LangServe: 将LangChain链部署为REST API。
- LangSmith: 一个开发者平台，调试、测试、评估和监控LLM应用程序。

LangChain简化了LLM应用程序生命周期的各个阶段：

- 开发阶段：使用LangChain的开源构建块和组件构建应用程序，利用第三方集成和模板快速启动。
- 生产化阶段：使用LangSmith检查、监控和评估您的链，从而可以自信地持续优化和部署。
- 部署阶段：使用LangServe将任何链转化为API。

以开发阶段为例，在开发大模型RAG应用时，所有和应用相关的外部系统都能用上LangChain中的组件



Langchain的核心组件

LangChain 1.0中组件其实相当多的，核心组件主要有七大部分：

Core components核心组件

Agents代理

Models模型

Messages消息

Tools工具

Short-term memory短期记忆

Streaming流

Structured output结构化输出

1. 代理 (Agents)

在 v1.0 中，Agent 是应用的核心构建块。

- **新特性**：引入了统一的 `create_agent` 接口，取代了以往繁杂的 `AgentExecutor`。
- **核心逻辑**：Agent 本质上是一个“循环”，它接收用户输入，决定是否调用工具，处理工具结果，并重复此过程直到完成任务。
- **底层驱动**：默认基于 `LangGraph` 运行，这意味着所有的 Agent 现在都原生支持状态管理、分支逻辑和持久化，不仅能处理简单的问答，还能处理复杂的长流程任务。

2. 模型 (Models)

模型是 Agent 的“大脑”，v1.0 对其进行了高度标准化。

- **标准内容块：**解决了不同模型（如 OpenAI, Anthropic, Gemini）输出格式不一致的痛点。无论底层使用哪个模型，LangChain 都会将其输出（文本、工具调用、推理过程）统一为标准格式，方便开发者无缝切换模型。
- **Chat Models：**v1.0 推荐全面使用 Chat Model 接口（而非纯文本 LLM），以支持多模态输入和结构化交互。

3. 消息 (Messages)

消息是模型交互的基础数据单元，v1.0 对其结构进行了严格规范。

- **核心类型：**
 - `HumanMessage`：用户输入。
 - `AIMessage`：模型回复（包含思维过程 `thought_process` 和工具调用 `tool_calls`）。
 - `SystemMessage`：设定 Agent 的行为准则和角色。
 - `ToolMessage`：工具执行后的返回结果。
- **作用：**Agent 的“短期记忆”本质上就是由这些消息组成的列表。

4. 工具 (Tools)

工具是 Agent 连接外部世界（API、数据库、搜索引擎）的桥梁。

- **定义方式：**使用 `@tool` 装饰器轻松将 Python 函数转换为 Agent 可用的工具。
- **主要改进：**v1.0 优化了工具调用的错误处理机制。如果 Agent 调用工具时参数填错，系统会自动捕获错误并反馈给 Agent，让其自我修正并重试，无需人工干预。

5. 短期记忆 (Short-term Memory)

在 v1.0 (结合 LangGraph) 中，记忆机制发生了根本性变化。

- **旧版 vs 新版：**不再使用旧式的 `ConversationBufferMemory` 类。
- **线程级持久化：**现在的短期记忆通过 `Checkpoint` 机制实现。
 - 系统会自动保存 Agent 在每一步的状态。
 - 通过传入 `thread_id`，Agent 可以随时加载之前的对话上下文，继续之前的任务。

6. 流 (Streaming)

LangChain 的流式系统允许将 Agent 运行过程中的实时反馈呈现给你的应用程序。

- **流式 Token：**最基础的“打字机效果”，实时显示模型生成的每一个字。
- **流式进度：**实时展示 Agent 的执行状态。

- **流式自定义更新**：允许开发者手动发送业务信号（例如：发送“已下载 10/100 条数据”这样的进度提示）。

7. 结构化输出 (Structured Output)

- **机制**：LangChain 的 `create_agent` 自动处理结构化输出，通过 **`response_format`** 参数直接配置（如传入 Pydantic 类）。
- **结果**：当模型生成结构化数据时，会被捕获、验证，并以代理状态的“`structured_response`”键返回。

LangChain应用程序生命周期

LLM 应用程序生命周期有三个阶段：开发、测试、生产部署，现在我们就通过一个实际的LangChain项目来学习LangChain在这三个阶段的运用，并且学习到 Model I/O、链。

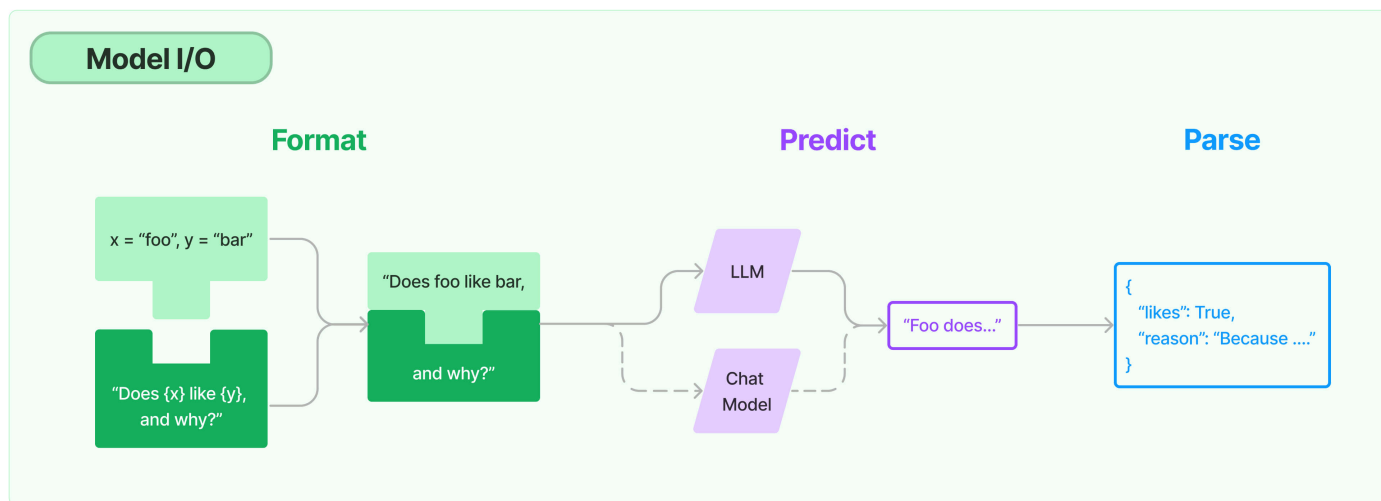
!! 在v1.0中链虽已不被推荐使用，但是作为langchain 的一种重要思想是有必要学习的。

LangChain之Model I/O

可以把对模型的使用过程拆解成三块：输入提示(Format)、调用模型(Predict)、输出解析(Parse)

- 1.提示模板: LangChain的模板允许动态选择输入，根据实际需求调整输入内容，适用于各种特定任务和应用。
- 2.语言模型: LangChain 提供通用接口调用不同类型的语言模型，提升了灵活性和使用便利性。
- 3.输出解析: 利用 LangChain 的输出解析功能，精准提取模型输出中所需信息，避免处理冗余数据，同时将非结构化文本转换为可处理的结构化数据，提高信息处理效率。

这三块形成了一个整体，在LangChain中这个过程被统称为Model I/O。针对每块环节，LangChain都提供了模板和工具，可以帮助快捷的调用各种语言模型的接口



Model 模型

LangChain支持的模型有三大类

- 1.大语言模型（LLM）
- 2.聊天模型（Chat Model）
- 3.文本嵌入模型（Embedding Model），这些模型将文本作为输入并返回词向量。严格来说，该模型不属于模型包装器，在后面的课程会单独学习和使用。

大语言模型（LLM），也叫文本补全模型，接收文本字符串作为输入，并返回一个补全字符串作为输出。

例如给定一个提示“今天的天气如何？”模型会生成一个相应的答案“今天的天气很好。”

可以理解为只能一问一答。

OpenAI的文本补全类型API在2023年7月进行了最后一次更新，该API现在只能用于访问较旧的历史遗留模型。而且目前国内的大模型厂商普遍对该API也未提供支持。所以该包装器我们不予学习和考虑。

聊天模型

聊天模型（Chat Model），主要代表Open AI的ChatGPT系列模型。这些模型通常由语言模型支持，但它们的API更加结构化。聊天模型包装器为 ChatOpenAI。

具体来说，ChatOpenAI将一系列的聊天消息/聊天消息列表作为输入，并返回聊天消息。兼容openAI的模型都可用ChatOpenAI包装器。不是兼容openAI的模型，使用 `init_chat_model` 创建，兼容任何的大模型。

聊天模型包装器的设计目标是处理复杂的对话场景，它支持多轮对话。这个聊天消息列表中的每一则消息都包含了消息角色和消息内容，它们合在一起构成了一个完整的对话上下文。这种输入方式非常适合处理那些需要引入历史对话内容以便生成带有对话上下文的响应的任务。

这也是目前国内的大模型厂商普遍支持的API。用一个实际的例子来看看聊天模型包装器的使用。

代码块

```
1  #学会通过LangChain 调用大模型，实现交互
2  import os
3
4  from langchain_openai import ChatOpenAI
5
6  from models import Constants, ALI_TONGYI_API_KEY_OS_VAR_NAME, ALI_TONGYI_URL,
   ALI_TONGYI_MAX_MODEL, \
7      TENCENT_HUNYUAN_API_KEY_OS_VAR_NAME, TENCENT_HUNYUAN_URL,
   TENCENT_HUNYUAN_TURBO_MODEL, get_lc_model_client
8
9  #获得访问大模型的key，该key保存在操作系统的环境变量中
10 MODEL_API_KEY = os.getenv("DASHSCOPE_API_KEY")
11 #访问大模型的客户端
```



```

12 client = ChatOpenAI(api_key=MODEL_API_KEY,base_url=
    "https://dashscope.aliyuncs.com/compatible-mode/v1"
13                        ,model="qwen-max-latest")
14
15 # MODEL_API_KEY = os.getenv(ALI_TONGYI_API_KEY_OS_VAR_NAME)
16 # #访问大模型的客户端
17 # client = ChatOpenAI(api_key=MODEL_API_KEY,base_url= ALI_TONGYI_URL
18 #                        ,model=ALI_TONGYI_MAX_MODEL)
19
20 # MODEL_API_KEY = os.getenv(TENCENT_HUNYUAN_API_KEY_OS_VAR_NAME)
21 # #访问大模型的客户端
22 # client = ChatOpenAI(api_key=MODEL_API_KEY,base_url= TENCENT_HUNYUAN_URL
23 #                        ,model=TENCENT_HUNYUAN_TURBO_MODEL)
24
25 # client = get_lc_model_client()
26 # client =
    get_lc_model_client(os.getenv(ALI_TONGYI_API_KEY_OS_VAR_NAME),ALI_TONGYI_URL,AL
    I_TONGYI_MAX_MODEL)
27 '''
28 注意，因为上面的两行代码在每个程序中都几乎会用到，
29 所以被抽取出来放到了models.py中，成为了函数get_lc_model_client
30 get_lc_model_client已经设定了缺省大模型，也允许传入参数自行设定模型
31 在后面的代码中则会直接使用该函数来获得访问大模型的客户端，不再另外说明
32 '''
33 msg = [
34     ('system','请将以下的内容翻译成英文'),
35     ('human','你好，你今天过得好吗? '),
36 ]
37
38 #原始答复比较复杂，包含了很多额外的信息
39 result = client.invoke(msg)
40 print(result)
41
42

```

从代码运行可以看到，虽然我们和大模型能正常的交互，但是有两个问题：

- 1、提示词是固定，无法根据实际情况灵活变动，比如“英语”想换成“意大利文”，“你好，你今天过得好吗？”换成其他文字
- 2、大模型原始答复比较复杂，包含了很多额外的信息

```

msg = [
    ('system','请将以下的内容翻译成英文'),
    ('human','你好，你今天过得好吗? '),
]

```

```
content='Hello, how are you today?' additional_kwargs={'refusal': None} response_metadata={'token_usage': {'completion_tokens': 7, 'prompt_tokens': 18, 'total_tokens': 25, 'completion_tokens_details': None, 'prompt_tokens_details': None}, 'model_name': 'hunyuan-turbos-latest', 'system_fingerprint': '', 'id': '1fde12d2b69c505c1669858598040e80', 'finish_reason': 'stop', 'logprobs': None} id='run-b8ef0fb1-96a3-4413-a114-06d7b49de63d-0' usage_metadata={'input_tokens': 18, 'output_tokens': 7, 'total_tokens': 25, 'input_token_details': {}, 'output_token_details': {}}
```

要解决这两个问题，就需要分别使用提示词模板、输出解析器。

提示模板

在LangChain的Model I/O中，提示模板是其组成之一,语言模型的提示是用户提供的一组指令或输入，用于指导模型的响应，帮助模型理解上下文并生成相关且连贯的基于语言的输出，例如回答问题、完成句子或参与某项活动、对话

PromptTemplates 是LangChain中的一个概念，通过接收原始用户输入，并返回一个准备好传递给语言模型的信息（即提示词 prompt）

通俗点说，prompt template 是一个模板化的字符串，可以用来生成特定的提示（prompts）。你可以将变量插入到模板中，从而创建出不同的提示。这对于重复生成相似格式的提示非常有用，尤其是在自动化任务中

LangChain提示模板特点

1. 清晰易懂的提示: 提高提示文本的可读性，使其更易于理解，尤其是在处理复杂或涉及多个变量的情况下。
2. 增强可重用性: 使用模板，可以在多个地方重复使用，简化代码，无需重复构建提示字符串。
3. 简化维护: 使用模板后，如果需要更改提示内容，只需修改模板，无需逐个查找所有用到该提示的地方。
4. 智能处理变量: 模板可以自动处理变量的插入，无需手动拼接字符串。
5. 参数化生成: 模板可以根据不同的参数生成不同的提示，有助于个性化文本生成。

类型

1. LLM提示模板 PromptTemplate：常用的String提示模板
2. 聊天提示模板 ChatPromptTemplate： 常用的Chat提示模板，用于组合各种角色的消息模板，传入聊天模型。消息模板包括：ChatMessagePromptTemplate、HumanMessagePromptTemplate、AIMessagePromptTemplate、SystemMessagePromptTemplate等
3. 样本提示模板 FewShotPromptTemplate：通过示例来教模型如何回答
4. 提示模板部分格式化：适用于需要先给某些参数赋值，其余参数后期赋值。
5. 自定义模板：允许基于其他模板类来定制自己的提示模板。

创建提示模板

代码块

```
1 #使用PromptTemplate, 方法一
```

```

2  # 创建原始模板，其中用{}包含的部分是占位符，可以在运行时替换为实际值
3  template = "您是一位专业的程序员。\\n对于信息 {text} 进行简短描述"
4
5  # 根据原始模板创建LangChain提示模板
6  prompt = PromptTemplate.from_template(template)
7  # 实际值，可以是用户通过其他方式传入，比如网页上输入
8  fact_text = "langchain"
9  # 打印LangChain提示模板的内容
10 print(prompt)
11 print("="*50)
12 print(prompt.format(text=fact_text))

```

直接生成提示模版

代码块

```

1  #使用PromptTemplate，方法二
2  prompt2 = PromptTemplate(
3      input_variables=["text"],
4      template="您是一位专业的程序员。\\n对于信息 {text} 进行简短描述"
5  )
6  print(prompt2.format(text="langchain框架"))

```

提示模板和大模型结合起来

调用语言模型，让模型帮写答案，并返回结果。

代码块

```

1  #学会使用提示词模版
2  from langchain.messages import SystemMessage, HumanMessage
3  from langchain_core.output_parsers import StrOutputParser
4  from langchain_core.prompts import ChatPromptTemplate,
5  SystemMessagePromptTemplate, HumanMessagePromptTemplate, \
6  PromptTemplate
7
8  from models import get_lc_model_client
9
10 #调用大模型
11 client = get_lc_model_client()
12
13 #使用PromptTemplate，方法一
14 # 创建原始模板，其中用{}包含的部分是占位符，可以在运行时替换为实际值
15 template = "您是一位专业的程序员。\\n对于信息 {text} 进行简短描述"
16
17 # 根据原始模板创建LangChain提示模板

```

```

17  prompt = PromptTemplate.from_template(template)
18  # 实际值, 可以是用户通过其他方式传入, 比如网页上输入
19  fact_text = "langchain"
20  # 打印LangChain提示模板的内容
21  print(prompt)
22  print("="*50)
23  print(prompt.format(text=fact_text))
24
25  #使用PromptTemplate, 方法二
26  prompt2 = PromptTemplate(
27      input_variables=["text"],
28      template="您是一位专业的程序员。\\n对于信息 {text} 进行简短描述"
29  )
30  # 打印LangChain提示模板的内容
31  print(prompt2)
32  print("="*50)
33  print(prompt2.format(text="langchain框架"))
34
35  result = client.invoke(prompt.format(text=fact_text))
36  print(result)

```

ChatPromptTemplate聊天提示模板

PromptTemplate创建字符串提示的模板。默认情况下, 使用Python的str.format语法进行模板化。而ChatPromptTemplate是创建聊天消息列表的提示模板。

创建一个ChatPromptTemplate提示模板, 模板的不同之处是它们有对应的角色。

LangChain提供不同类型的MessagePromptTemplate.最常用的是AIMessagePromptTemplate、SystemMessagePromptTemplate和HumanMessagePromptTemplate, 分别创建人工智能消息、系统消息和人工消息。

代码块

```

1  #学会使用提示词模版
2  from langchain.messages import SystemMessage, HumanMessage
3  from langchain_core.output_parsers import StrOutputParser
4  from langchain_core.prompts import ChatPromptTemplate,
5      SystemMessagePromptTemplate, HumanMessagePromptTemplate, \
6      PromptTemplate
7  from models import get_lc_model_client
8
9  #调用大模型
10 client = get_lc_model_client()
11
12 #使用ChatPromptTemplate, 它是个大模版, 可以组合各种角色的消息模板

```

```

13  #定义提示模版
14  '''LangChain提供不同类型的角色的消息模板。
15  最常用的是AIMessagePromptTemplate、 SystemMessagePromptTemplate和
16  HumanMessagePromptTemplate，
17  分别创建代表人工智能应答消息、系统消息（提示词中的角色、任务、参考案例、约束项等等）和人类
18  消息（具体的任务）。'''
19  chat_template = ChatPromptTemplate.from_messages(
20      [
21          #改为 ('system', '请将以下的内容翻译成{language}') 也可以
22          SystemMessagePromptTemplate.from_template("请将以下的内容翻译成
23          {language}"),
24          # 改为 HumanMessagePromptTemplate.from_template("{text}") 也可以
25          ('human', '{text}')
26      ]
27  )
28  print(chat_template)
29  print(chat_template.format(language='英文', text='你好，今天的天空真蓝! '))
30  #原始答复比较复杂，包含了很多额外的信息
31  result = client.invoke(chat_template.format(language='英文', text='你好，今天的天
32  空真蓝! '))
33  print(result)
34
35  print(chat_template.format_messages(language='意大利文', text='朋友啊再见! '))
36  result = client.invoke(chat_template.format_messages(language='意大利文',
37  text='朋友啊再见! '))
38  print(result)
39
40  print(chat_template.format_prompt(language='法文', text='上帝保佑吾王'))
41  result = client.invoke(chat_template.format_prompt(language='法文', text='上帝保
42  佑吾王'))
43  print(result)

```

输出解析器（25.11.19更新）

输出解析器负责将 LLM 调用的输出解析为结构化数据。

LangChain有许多不同类型的输出解析器：CommaSeparatedListOutputParser、DatetimeOutputParser、JsonOutputParser、XMLOutputParser等等。

此外，在langchain V1.0中引入了 `content_blocks` 属性，可以以一种标准化的方式处理模型返回的文本、推理过程、工具调用等不同类型的內容。

代码块

```

1  #学会使用输出解析器
2  from langchain_core.output_parsers import StrOutputParser

```

```

3  from langchain_core.prompts import ChatPromptTemplate,
   SystemMessagePromptTemplate
4  from models import get_lc_model_client
5
6  #获得访问大模型客户端
7  client = get_lc_model_client()
8
9  #使用ChatPromptTemplate, 它是个大模版, 可以组合各种角色的消息模板
10 #定义提示模版
11 '''LangChain提供不同类型的角色的消息模板.
12 最常用的是AIMessagePromptTemplate、 SystemMessagePromptTemplate和
13 HumanMessagePromptTemplate,
14 分别创建代表人工智能应答消息、系统消息（提示词中的角色、任务、参考案例、约束项等等）和人类
15 消息（具体的任务）。'''
16 chat_template = ChatPromptTemplate.from_messages(
17     [
18         #改为 ('system', '请将以下的内容翻译成{language}') 也可以
19         SystemMessagePromptTemplate.from_template("请将以下的内容翻译成
20 {language}"),
21         # 改为 HumanMessagePromptTemplate.from_template("{text}") 也可以
22         ('human', '{text}')
23     ]
24 )
25
26 #原始答复比较复杂, 包含了很多额外的信息
27 result = client.invoke(chat_template.format(language='意大利文', text='朋友啊再
28 见! '))
29 print("----- 原始结果 -----")
30 print(result)
31
32 #2、解析返回结果
33 parser = StrOutputParser()
34 print("\n----- StrOutputParser()解析后的结果 -----")
35 print(parser.invoke(result))
36 print("\n----- content_blocks 属性解析 -----")
37 print(result.content_blocks)
38 for block in result.content_blocks:
39     if block["type"] == "text":
40         print(block['text'])

```

----- 原始结果 -----

```

content='Amico, arrivederci!' additional_kwargs={'refusal': None} response_metadata={'token_usage': {'completion_tokens': 7, 'prompt_tokens':
27, 'total_tokens': 34, 'completion_tokens_details': None, 'prompt_tokens_details': None}, 'model_provider': 'openai', 'model_name': 'qwen-max
-latest', 'system_fingerprint': None, 'id': 'chatcmpl-4688e347-58ae-42f3-a950-c9b3d5063925', 'finish_reason': 'stop', 'logprobs': None} id='lc
_run--9fe523b9-b976-4b63-a581-4bbf6af91c86-0' usage_metadata={'input_tokens': 27, 'output_tokens': 7, 'total_tokens': 34, 'input_token_details
': {}, 'output_token_details': {}}

```

----- StrOutputParser()解析后的结果 -----

```
Amico, arrivederci!
```

----- content_blocks 属性解析 -----

```

[{'type': 'text', 'text': 'Amico, arrivederci!'}]
Amico, arrivederci!

```

`content_blocks` 解决了不同大模型（OpenAI, Anthropic, Google 等）之间消息格式不统一的痛点。它在LangChain层将所有模型提供商异构的API响应，统一转换成标准化的内容块。这让开发者的上层应用代码与底层具体模型实现解耦，从而无需再为每个模型编写定制化的解析代码。

代码块

```
1 from langchain_community.chat_models import ChatTongyi
2 from models import get_ali_model_client
3
4 # 初始化模型
5 qwen_model = ChatTongyi(model_name="qwen-max")
6 deepseek_model = get_ali_model_client()
7
8 qwen_response = qwen_model.invoke("你是谁")
9 deepseek_response = deepseek_model.invoke("你是谁")
10
11 print("----- AMessage 的原始结构 -----")
12 print(qwen_response)
13 print("\n")
14 print(deepseek_response)
15 print("\n----- 解析 content_blocks -----")
16 print(qwen_response.content_blocks)
17 print("\n")
18 print(deepseek_response.content_blocks)
```

----- AMessage 的原始结构 -----

```
content='我是Qwen，由阿里云开发的预训练语言模型。我被设计用来帮助用户生成各种类型的文本，如文章、故事、诗歌、故事等，并能够根据不同的场景和需求提供信息和帮助。无论是创意写作还是解决实际问题，我都会尽力提供支持。有什么我可以帮你的吗？' additional_kwargs={} response_metadata={'model_name': 'qwen-max', 'finish_reason': 'stop', 'request_id': '18f54a44-dc3a-42f5-a09b-8f244af97f16', 'token_usage': {'input_tokens': 10, 'output_tokens': 66, 'prompt_tokens_details': {'cached_tokens': 0}, 'total_tokens': 76}, 'id': 'lc_run--7ab9889f-45fc-427c-833b-5a288fba84a6-0'}
```

```
content='我是DeepSeek Chat，由深度求索公司（DeepSeek）创造的智能AI助手！🌟 我可以回答你的问题、陪你聊天、帮你查找资料、提供各种学习和工作上的帮助。无论是科技、历史、娱乐，还是生活小窍门，都可以来问我哦！😊\n\n有什么我可以帮你的吗？' additional_kwargs={'refusal': None} response_metadata={'token_usage': {'completion_tokens': 69, 'prompt_tokens': 4, 'total_tokens': 73, 'completion_tokens_details': None, 'prompt_tokens_details': None}, 'model_provider': 'openai', 'model_name': 'deepseek-v3', 'system_fingerprint': None, 'id': 'chatcmpl-5bfe72e1-94e8-4ffa-8ba0-545dc03c13cc', 'finish_reason': 'stop', 'logprobs': None, 'id': 'lc_run--2d1668c9-87bd-4c81-a597-fad1f1f957a6-0', 'usage_metadata': {'input_tokens': 4, 'output_tokens': 69, 'total_tokens': 73, 'input_token_details': {}, 'output_token_details': {}}}
```

----- 解析 content_blocks -----

```
[{'type': 'text', 'text': '我是Qwen，由阿里云开发的预训练语言模型。我被设计用来帮助用户生成各种类型的文本，如文章、故事、诗歌、故事等，并能够根据不同的场景和需求提供信息和帮助。无论是创意写作还是解决实际问题，我都会尽力提供支持。有什么我可以帮你的吗？'}]
```

```
[{'type': 'text', 'text': '我是DeepSeek Chat，由深度求索公司（DeepSeek）创造的智能AI助手！🌟 我可以回答你的问题、陪你聊天、帮你查找资料、提供各种学习和工作上的帮助。无论是科技、历史、娱乐，还是生活小窍门，都可以来问我哦！😊\n\n有什么我可以帮你的吗？'}]
```

ChatTongyi和ChatOpenAI返回的原始响应是有区别，通过 `content_blocks` 把输出解析成统一的结构化内容。

再看一个工具调用的例子， `content_blocks` 会自动提取核心业务逻辑所需的信息。

代码块

```
1 import os
```



```

2  from langchain_community.chat_models import ChatTongyi
3  from langchain.tools import tool
4  from langchain.messages import HumanMessage
5  from models import get_ali_model_client
6
7  # 1. 定义一个或多个工具
8  @tool
9  def get_current_weather(location: str, unit: str = "celsius") -> str:
10     """获取指定地点的当前天气信息。"""
11     # 这是一个模拟函数，实际应用中会调用真实的天气API
12     if "北京" in location:
13         return f"北京今天天气晴朗，气温为 25 度 {unit}。"
14     elif "上海" in location:
15         return f"上海今天有小雨，气温为 20 度 {unit}。"
16     else:
17         return f"抱歉，无法查询 {location} 的天气。"
18
19  # 2. 初始化模型
20  # 请使用支持函数调用/工具调用的模型
21  model = ChatTongyi(model_name="qwen-plus")
22  deepseek_model = get_ali_model_client()
23
24  # 3. 将工具绑定到模型
25  model_with_tools = model.bind_tools([get_current_weather])
26  deepseek_model_with_tools = deepseek_model.bind_tools([get_current_weather])
27
28  # 4. 调用模型，并传入一个需要使用工具的提示
29  prompt = "今天北京的天气怎么样？"
30  ai_msg = model_with_tools.invoke([HumanMessage(content=prompt)])
31  deepseek_ai_msg =
32  deepseek_model_with_tools.invoke([HumanMessage(content=prompt)])
33
34  # 5. 使用 content_blocks 来解析模型的响应
35  print("----- AIMessage 的原始结构 -----")
36  print(ai_msg)
37  print("\n")
38  print(deepseek_ai_msg)
39  print("\n----- 解析 content_blocks -----")
40  print(ai_msg.content_blocks)
41  print("\n")
42  print(deepseek_ai_msg.content_blocks)
43
44  # 统一访问不同类型的内容块
45  # if not ai_msg.content_blocks:
46  #     print("模型没有返回任何 content_blocks。")
47  # else:
48  #     for block in ai_msg.content_blocks:

```

```

48 #         if block["type"] == "reasoning":
49 #             print(f"推理过程: {block['reasoning']}")
50 #         elif block["type"] == "text":
51 #             print(f"回答内容: {block['text']}")
52 #         elif block["type"] == "tool_call":
53 #             print(f"模型请求调用工具: {block['name']}")
54 #             print(f"工具所需的参数: {block['args']}")

```

----- AMessage 的原始结构 -----

```

content=' additional_kwargs={'tool_calls': [{'index': 0, 'id': 'call_7e17f237eb6942cdb4ad62', 'type': 'function', 'function':
{'name': 'get_current_weather', 'arguments': {'location': '北京'}}]} response_metadata={'model_name': 'qwen-plus', 'finish
_reason': 'tool_calls', 'request_id': 'e89a5b50-d8db-4630-9f1c-711303568ca9', 'token_usage': {'input_tokens': 171, 'output_tok
ens': 20, 'total_tokens': 191, 'prompt_tokens_details': {'cached_tokens': 0}}} id='lc_run--6d60622a-058b-40c3-a44b-7192b0b5899
3-0' tool_calls=[{'name': 'get_current_weather', 'args': {'location': '北京'}, 'id': 'call_7e17f237eb6942cdb4ad62', 'type': 't
ool_call'}]

```

```

content='' additional_kwargs={'refusal': None} response_metadata={'token_usage': {'completion_tokens': 25, 'prompt_tokens': 16
2, 'total_tokens': 187, 'completion_tokens_details': None, 'prompt_tokens_details': None}, 'model_provider': 'openai', 'model_
name': 'deepseek-v3', 'system_fingerprint': None, 'id': 'chatcmpl-0259da46-9793-4550-9cee-c7d13f4e12ce', 'finish_reason': 'too
l_calls', 'logprobs': None} id='lc_run--06142792-8605-450b-a958-058e45118052-0' tool_calls=[{'name': 'get_current_weather', 'a
rgs': {'location': '北京', 'unit': 'celsius'}, 'id': 'call_c006eebed174412f91e62d', 'type': 'tool_call'}] usage_metadata={'inp
ut_tokens': 162, 'output_tokens': 25, 'total_tokens': 187, 'input_token_details': {}, 'output_token_details': {}}

```

----- 解析 content_blocks -----

```

[{'type': 'tool_call', 'id': 'call_7e17f237eb6942cdb4ad62', 'name': 'get_current_weather', 'args': {'location': '北京'}}]

```

```

[{'type': 'tool_call', 'name': 'get_current_weather', 'args': {'location': '北京', 'unit': 'celsius'}, 'id': 'call_c006eebed17
4412f91e62d'}]

```

链

如何将前面的提示词模版、模型包装器和输出解析器串起来，这时可以使用链。

链可以将模型输入输出整合在一个链中操作。它利用提示模板格式化输入，将格式化后的字符串传递给LLM模型，并返回LLM的输出。这样使得整个处理过程更加高效和便捷。



代码块

```

1 #学会使用链
2 from langchain_core.output_parsers import StrOutputParser
3 from langchain_core.prompts import ChatPromptTemplate,
  SystemMessagePromptTemplate
4
5 from models import get_lc_model_client
6
7 #获得访问大模型客户端
8 client = get_lc_model_client()
9
10 #使用ChatPromptTemplate, 它是个大模版, 可以组合各种角色的消息模板
11 #定义提示模版
12 '''LangChain提供不同类型的角色的消息模板.

```

```

13  最常用的是AIMessagePromptTemplate、 SystemMessagePromptTemplate和
    HumanMessagePromptTemplate，
14  分别创建代表人工智能应答消息、系统消息（提示词中的角色、任务、参考案例、约束项等等）和人类
    消息（具体的任务）。'''
15  chat_template = ChatPromptTemplate.from_messages(
16      [
17          #改为 ('system','请将以下的内容翻译成{language}') 也可以
18          SystemMessagePromptTemplate.from_template("请将以下的内容翻译成
    {language}"),
19          # 改为 HumanMessagePromptTemplate.from_template("{text}") 也可以
20          ('human', '{text}')
21      ]
22  )
23
24  # #原始答复比较复杂，包含了很多额外的信息
25  # result = client.invoke(chat_template.format(language='意大利文', text='朋友啊再
    见! '))
26  # print(result)
27
28  # #2、解析返回结果
29  parser = StrOutputParser()
30  # print(parser.invoke(result))
31
32  chain = chat_template | client | parser
33  chain.invoke({'language': '意大利文', 'text': '朋友啊再见! '})

```

跟踪和调试

在开发应用程序时，如何跟踪LangChain中各个组件是否能正常工作呢？

方法一：使用LangSmith

LangSmith是LangChain提供的一个跟踪和评估大语言模型应用程序的组件、框架、平台，个人开发使用LangSmith一定额度内免费，企业使用需要收费，所以我们不予考虑

方法二：开启LangChain应用程序的调试功能

代码块

```

1  #学会LangChain应用程序的跟踪调试
2  import langchain
3  from langchain_core.output_parsers import StrOutputParser
4  from langchain_core.prompts import ChatPromptTemplate,
    SystemMessagePromptTemplate
5
6  from models import get_lc_model_client
7
8  #开启该参数，会输出调试信息

```

```

9  langchain.debug = True
10 #获得访问大模型客户端
11 client = get_lc_model_client()
12
13 #使用ChatPromptTemplate, 它是个大模版, 可以组合各种角色的消息模板
14 #定义提示模版
15 '''LangChain提供不同类型的角色的消息模板.
16 最常用的是AIMessagePromptTemplate、 SystemMessagePromptTemplate和
17 HumanMessagePromptTemplate,
18 分别创建代表人工智能应答消息、系统消息（提示词中的角色、任务、参考案例、约束项等等）和人类
19 消息（具体的任务）。'''
20 chat_template = ChatPromptTemplate.from_messages(
21     [
22         #改为 ('system', '请将以下的内容翻译成{language}') 也可以
23         SystemMessagePromptTemplate.from_template("请将以下的内容翻译成
24 {language}"),
25         # 改为 HumanMessagePromptTemplate.from_template("{text}") 也可以
26         ('human', '{text}')
27     ]
28 )
29
30 # #原始答复比较复杂, 包含了很多额外的信息
31 # result = client.invoke(chat_template.format(language='意大利文', text='朋友啊再
32 见! '))
33 # print(result)
34
35 # #2、解析返回结果
36 parser = StrOutputParser()
37 # print(parser.invoke(result))
38
39 chain = chat_template | client | parser
40 print(chain.invoke({'language': '意大利文', 'text': '朋友啊再见! '}))

```

发布应用程序

在应用程序中增加相应的服务部署代码即可

代码块

```

1  #学会利用LangChain部署我们的应用成为WEB服务
2  from fastapi import FastAPI
3
4  from langchain_core.output_parsers import StrOutputParser
5  from langchain_core.prompts import ChatPromptTemplate,
6  SystemMessagePromptTemplate, HumanMessagePromptTemplate
7  from langserve import add_routes

```

```

7
8 from models import get_lc_model_client
9
10 #获得访问大模型客户端
11 client = get_lc_model_client()
12
13 #解析返回结果
14 parser = StrOutputParser()
15
16 #定义提示模版
17 prompt_template = ChatPromptTemplate.from_messages(
18     [
19         #改为 ('system', '请将以下的内容翻译成{language}') 也可以
20         SystemMessagePromptTemplate.from_template("请将以下的内容翻译成
21 {language}"),
22         # 改为 HumanMessagePromptTemplate.from_template("{text}") 也可以
23         ('human', '{text}')
24     ]
25 )
26 # 以链的形式调用
27 chain = prompt_template | client | parser
28
29 #部署为服务
30 app = FastAPI(title="基于LangChain的服务",version="V1.5",description="翻译服务")
31 add_routes(app, chain,path="/tslServer")
32
33 if __name__ == "__main__":
34     import uvicorn
35     uvicorn.run(app, host="localhost", port=8000)

```

之后就可以利用类似ApiFox、PostMan之类的工具通过api接口进行访问，自然也可以通过其他编程语言通过api接口进行访问

POST http://localhost:8000/tslServer/invoke

请求 响应定义 接口说明 预览文档 Mock 未命名接口

Params Body 1 Headers 8 Cookies Auth 前置操作 后置操作 设置

none form-data x-www-form-urlencoded json xml raw binary GraphQL msgpack

参数值 数据结构

```
1 {
2   "input":
3   {
4     "language": "英语",
5     "text": "你好，今天的天空真蓝！"
6   }
7 }
```

Body Cookie Header 4 控制台 实际请求

Pretty Raw Preview Visualize JSON utf8

```
1 {
2   "output": "Hello, the sky is really blue today!",
3   "metadata": {
4     "run_id": "eee2a0a3-ad4d-4a08-8ea3-7aa7cd9065a2",
5     "feedback_tokens": []
6   }
7 }
```

如果使用LangChain本身作为客户端框架，访问这个服务则会更简单

代码块

```
1 #使用LangChain编写客户端访问我们基于LangServer的WEB服务
2 #对于其他编程语言来说，可以使用RESTful API来调用我们的服务
3 '''比如在postman或者apifox中访问http://localhost:8000/tslServer/invoke
4 在body中选择json，然后输入
5 {
6     "input":
7     {
8         "language": "意大利文",
9         "text": "为了部落！"
10     }
11 }'''
12 from langserve import RemoteRunnable
13
14 if __name__ == "__main__":
15     client = RemoteRunnable("http://localhost:8000/tslServer")
16     print(client.invoke({'language': '意大利文', 'text': '为了部落!'}))
```

增强提示词模版和输出解析器

增强提示词模板

前面我们已经看到了提示词模版的使用，当然我们知道，提示词编写有很多技巧，比如少样本提示等等，直接写在提示词里当然是可以的。但是LangChain为这些技巧也提供了相关的提示词模版

少量样本示例的提示模板

使用FewShotPromptTemplate或FewShotChatMessagePromptTemplate，两者使用基本一致

创建示例集：创建一些提示样本，每个示例都是一个字典，其中键是输入变量，值是输入变量的值

代码块

```
1 examples = [  
2     {"input": "2+2", "output": "4", "description": "加法运算"},  
3     {"input": "5-2", "output": "3", "description": "减法运算"},  
4 ]
```

创建提示模板

代码块

```
1 from langchain.prompts import PromptTemplate  
2  
3 # 创建提示模板，配置一个提示模板，将一个示例格式化为字符串  
4 prompt_template = "你是一个数学专家,算式: {input} 值: {output} 使用:  
5 {description} "  
6  
7 # 这是一个提示模板，用于设置每个示例的格式  
8 prompt_sample = PromptTemplate.from_template(prompt_template)  
9  
10 # input="2+2", output="4", description="加法运算"  
11 # print(prompt_sample.format_prompt(**examples[1]))  
12 # print(prompt_sample.format_prompt(input="2+2", output="4", description="加法运算"))
```

创建FewShotPromptTemplate对象

代码块

```
1 # 创建一个FewShotPromptTemplate对象  
2 from langchain.prompts.few_shot import FewShotPromptTemplate  
3  
4 prompt = FewShotPromptTemplate(  
5     examples=examples,  
6     prefix=prompt_template,  
7     suffix=""  
8 )
```



```

5     examples=examples,
6     example_prompt=prompt_sample,
7     suffix="你是一个数学专家,算式: {input} 值: {output}",
8     input_variables=["input", "output"]
9 )
10 print(prompt.format(input="2*5", output="10")) # 你是一个数学专家,算式: 2*5 值:
10

```

初始化大模型，然后调用

代码块

```

1  from langchain_openai import ChatOpenAI
2
3  model = ChatOpenAI(model_name='gpt-3.5-turbo')
4  result = model.invoke(prompt.format(input="2*5", output="10"))
5  print(result) # 使用: 乘法运算

```

代码 fewshot_prmpt.py

代码块

```

1  #少样本提示模版的使用
2  from langchain.prompts import PromptTemplate
3  from langchain.prompts.few_shot import FewShotPromptTemplate
4  from models import get_lc_model_client
5
6  #获得访问大模型客户端
7  client = get_lc_model_client()
8
9  # 创建示例
10 examples = [
11     {"input": "2+2", "output": "4", "description": "加法运算"},
12     {"input": "5-2", "output": "3", "description": "减法运算"},
13 ]
14
15 # 配置一个提示模板，用来一个示例格式化
16 examples_prompt_tmplt_txt = "算式: {input} 值: {output} 类型: {description} "
17
18 # 这是一个提示模板的实例，用于设置每个示例的格式
19 prompt_sample = PromptTemplate.from_template(examples_prompt_tmplt_txt)
20
21 # 创建少样本示例的对象
22 prompt = FewShotPromptTemplate(
23     examples=examples,
24     example_prompt=prompt_sample,

```

```

25     prefix="你是一个数学专家，能够准确说出算式的类型，",
26     suffix="现在给你算式：{input}，值：{output}，告诉我类型：",
27     input_variables=["input", "output"]
28 )
29 print(prompt.format(input="2*5", output="10")) # 你是一个数学专家,算式: 2*5 值:
10
30
31 print('-' * 50)
32
33 result = client.invoke(prompt.format(input="2*5", output="10"))
34 print(result.content) # 使用: 乘法运算
35

```

提示模板部分格式化

适用于需要先给某些参数赋值，其余参数后期赋值

代码 partial_prmpt.py

代码块

```

1  #提示模板部分格式化：适用于需要先给某些参数赋值，其余参数后期赋值。
2  #非常适用于动态获得特定变量值的情况，比如日期和时间，某些系统内部配置。这些值一般是无需
   用户在每次提问时输入。
3  #比如，实现一个学习助手，用户在提问前，先确定要问历史问题，地理问题等等
4  from langchain.prompts import PromptTemplate
5  from models import get_lc_model_client
6
7  #获得访问大模型客户端
8  client = get_lc_model_client()
9
10 def get_datetime():
11     import datetime
12     return datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
13
14 # 配置一个提示模板，用来一个示例格式化
15 prompt_tmplt_txt = "讲一个关于{date}的{story_type}"
16
17 prompt = PromptTemplate(
18     template=prompt_tmplt_txt,
19     input_variables=["date", "story_type"]
20 )
21 half_prompt = prompt.partial(date=get_datetime())
22 print(half_prompt)
23
24 result = client.invoke(half_prompt.format(story_type="笑话"))
25 print(result.content)

```

```
26 result = client.invoke(half_prompt.format(story_type="悲伤故事"))
27 print(result.content)
28
```

输出解析器

输出解析器负责获取 LLM 的输出并将其转换为更合适的格式。借助LangChain的输出解析器重构程序，使模型能够生成结构化回应，并可以直接解析这些回应

LangChain有许多不同类型的输出解析器

- CSV解析器:CommaSeparatedListOutputParser,模型的输出以逗号分隔，以列表形式返回输出
- 日期时间解析器:DatetimeOutputParser,可用于将 LLM 输出解析为日期时间格式
- JSON解析器:JsonOutputParser,确保输出符合特定JSON对象格式。
- XML解析器:XMLOutputParser,允许以流行的XML格式从LLM获取结果

各个输出解析器的基本使用

代码块

```
1  from langchain_core.prompts import ChatPromptTemplate
2  from langchain_openai import ChatOpenAI
3
4  # 初始化语言模型
5  model = ChatOpenAI(model_name="gpt-3.5-turbo", temperature=0.0)
6
7  # 创建解析器
8  from langchain_core.output_parsers import JsonOutputParser, StrOutputParser
9
10 # output_parser = StrOutputParser()
11 output_parser = JsonOutputParser()
12
13 # 提示模板
14 prompt = ChatPromptTemplate.from_messages([
15     ("system", "你是一个专业的程序员"),
16     ("user", "{input}")
17 ])
18
19 # 将提示和模型合并以进行调用
20 chain = prompt | model | output_parser
21
22 res = chain.invoke({"input": "langchain是什么? 问题用question 回答用ans 返回一个JSON格式"})
23 # res = chain.invoke({"input": "大模型中的langchain是什么?"})
24 print(res)
```

代码块

```
1  from langchain.output_parsers import DatetimeOutputParser
2  from langchain.prompts import PromptTemplate
3  from models import get_lc_model_client
4
5  # 获得访问大模型客户端
6  client = get_lc_model_client()
7
8  # 定义模板格式
9  template = """
10  回答用户的问题: {question}
11
12  {format_instructions}
13  """
14
15  # 使用日期时间解析器
16  output_parser = DatetimeOutputParser()
17
18  prompt = PromptTemplate.from_template(
19      template,
20      partial_variables={"format_instructions":
21          output_parser.get_format_instructions()},
22  )
23  # print(prompt)
24  # print(output_parser.get_format_instructions())
25  """
26  Write a datetime string that matches the following pattern: '%Y-%m-
27  %dT%H:%M:%S.%fZ'.
28
29  Examples: 0797-06-01T18:31:46.253315Z, 1236-04-01T03:29:24.290741Z, 1949-08-
30  11T00:31:18.305832Z
31
32  Return ONLY this string, no other words!
33  """
34
35  # 链式调用
36  chain = prompt | client | output_parser
37  # 执行
38  output = chain.invoke({"question": "新中国是什么时候成立的? "})
39  # 打印输出
40  print(output) # 1949-10-01
```

代码块

```
1  from langchain_core.prompts import ChatPromptTemplate
```

```

2  from langchain_core.output_parsers import CommaSeparatedListOutputParser,
   StrOutputParser
3  from models import get_lc_model_client
4
5  # 获得访问大模型客户端
6  client = get_lc_model_client()
7
8  # 创建解析器
9  #output_parser = StrOutputParser()
10 output_parser = CommaSeparatedListOutputParser()
11
12 # 提示模板
13 prompt = ChatPromptTemplate.from_messages([
14     ("system", "你是一个专业的程序员"),
15     ("user", "{input}")
16 ])
17
18 # 将提示和模型合并以进行调用
19 chain = prompt | client | output_parser
20
21 # 示例调用
22 print(chain.invoke({"input": "列出Python的三个主要版本，用逗号分隔"}))
23 print(chain.invoke({"input": "列举三个常见的机器学习框架，用逗号分隔"}))

```

Langchain的链和LCEL

!! 虽然LangChain v1.0已经移除了各种Chain，但是LCEL作为langchain的一种思想还是有必要去学习的

实际上，LangChain的名字源自其框架的核心设计思路：

用链(Chain)，将大语言模型开发的各个组件链接起来，以构建复杂的应用程序。

链的主要功能是管理应用程序中的数据流动，它将不同的组件(或其他链)链接在一起，形成一个完整的数据处理流程。每个组件都是链中的一个环节，它们按照预设的顺序，接力完成各自的任务，就好比工厂里的流水线。

LangChain早期的链只有一种实现方式，但是后来LangChain引入了LCEL链逐渐代替原有的实现。

所以在V0.1以后，开始对链进行了分类，分为了遗留链和LCEL链。在LangChain官方将所有的旧链全部替换为LCEL链之前，遗留链暂时可用。

详情可见API地址：https://python.langchain.com/api_reference/langchain/chains.html

所以我们以LCEL链为主要学习对象，如无特殊情况，遗留链不会进行说明和学习。

LCEL 也被称为LangChain表达式(LangChain Expression Language)，是一种用声明式的方法来链接LangChain组件。

所有可以被链起来的组件，如大语言模型、输出解析器、检索器、提示词模板等都支持下面三个方法：

stream: 流式返回响应的块

invoke: 接受输入返回输出

batch: 接受批量输入返回输出列表

所以最终组成的链在使用上，也是调用这三个方法。

链的基本使用

如何把组件链起来？

最简单最常见的就是用管道操作符 “|”，在前面的代码中，我们已经多次使用过了

```
chain = chat_template | client | parser
chain.invoke({'language': '意大利文', 'text': '朋友啊再见! '})
```

或者使用RunnableSequence

代码块

```
1  #学会使用链
2  from langchain_core.output_parsers import StrOutputParser
3  from langchain_core.prompts import ChatPromptTemplate,
   SystemMessagePromptTemplate
4  from langchain_core.runnables import RunnableSequence
5  from models import get_lc_model_client
6
7  #获得访问大模型客户端
8  client = get_lc_model_client()
9
10 #使用ChatPromptTemplate, 它是个大模版, 可以组合各种角色的消息模板
11 #定义提示模版
12 '''LangChain提供不同类型的角色的消息模板.
13 最常用的是AIMessagePromptTemplate、 SystemMessagePromptTemplate和
   HumanMessagePromptTemplate,
14 分别创建代表人工智能应答消息、系统消息（提示词中的角色、任务、参考案例、约束项等等）和人类
   消息（具体的任务）。'''
15 chat_template = ChatPromptTemplate.from_messages(
16     [
17         #改为 ('system', '请将以下的内容翻译成{language}') 也可以
18         SystemMessagePromptTemplate.from_template("请将以下的内容翻译成
   {language}"),
19         # 改为 HumanMessagePromptTemplate.from_template("{text}") 也可以
```

```

20         ('human', '{text}')
21     ]
22 )
23
24 # #2、解析返回结果
25 parser = StrOutputParser()
26
27 chain = RunnableSequence(chat_template, client, parser)
28 chain.invoke({'language': '意大利文', 'text': '朋友啊再见! '})

```

LCEL高级特性与组件

RunnableLambda

很多的时候LangChain组件未提供的功能，需要我们自己写函数实现，但是这个函数我们也需要把它加入链，作为工作流程的一部分，此时就需要使用RunnableLambda

代码块

```

1  #RunnableLambda的使用
2  from operator import itemgetter
3
4  from langchain_core.prompts import ChatPromptTemplate
5  from langchain_core.runnables import RunnableLambda
6  from langchain_core.runnables import chain
7  from models import get_lc_model_client
8
9  #获得访问大模型客户端
10 client = get_lc_model_client()
11
12 #定义提示模版
13 chat_template = ChatPromptTemplate.from_template(" {a} + {b}是多少? ")
14
15 #获得字符串的长度
16 def length_function(text):
17     return len(text)
18
19 #将两个字符串长度的数量相乘
20 def _multiple_length_function(text1, text2):
21     return len(text1) * len(text2)
22
23 @chain
24 def multiple_length_function(_dict):
25     return _multiple_length_function(_dict["text1"], _dict["text2"])
26
27 chain1 = chat_template | client

```

```

28  #使用RunnableLambda将函数转换为与LCEL兼容的组件
29  chain = (
30      {
31          "a": itemgetter("foo") | RunnableLambda(length_function),
32          "b": {"text1": itemgetter("foo"), "text2": itemgetter("bar")}
33              | multiple_length_function,
34      }
35      | chain1
36  )
37  print(chain.invoke({"foo": "abc", "bar": "abcd"}))

```

RunnableParallel

把组件链起来，前面我们使用了RunnableSequence。Sequence的中文意思“顺序，一连串”，也就是所有的组件或多个任务都是按顺序一个个执行的。

但是有时候，我们希望多个任务能够同时执行，这时可以使用RunnableParallel

代码块

```

1  #RunnableParallel的使用
2  from langchain_core.runnables import RunnableLambda, RunnableParallel
3
4  def add_one(x: int) -> int:
5      return x + 1
6
7  def mul_two(x: int) -> int:
8      return x * 2
9
10 def mul_three(x: int) -> int:
11     return x * 3
12
13 runnable_1 = RunnableLambda(add_one)
14 runnable_2 = RunnableLambda(mul_two)
15 runnable_3 = RunnableLambda(mul_three)
16
17 chain_seq = runnable_1 | runnable_2 | runnable_3
18 print(chain_seq.invoke(1))
19
20 chain = runnable_1 | RunnableParallel(
21     mul_two=runnable_2,
22     mul_three=runnable_3,
23 )
24
25 print(chain.invoke(1))

```


执行结果如下：

```
12
{'mul_two': 4, 'mul_three': 6}
```

可以看到使用了`chain_seq = runnable_1 | runnable_2 | runnable_3`的打印结果是12：

`chain_seq.invoke(1)`传入1，通过3个函数依次计算： $1+1=2 \rightarrow 2*2=4 \rightarrow 4*3=12$

但是使用了`RunnableParallel`的`chain`，结果的来源是： $1+1=2 \rightarrow 2*2=4$ 以及 $1+1=2 \rightarrow 2*3=6$

RunnablePassthrough

`RunnablePassthrough`类允许我们在`LangChain`的链中传递数据：

1、确保数据保持不变，直接用于后续步骤的输入。`RunnablePassthrough`常用在链的第一个位置，用于接收用户的输入（也可以用在中间位置，则用于接收上一步的输出）。

2、`RunnablePassthrough`也允许通过`assign`对数据增强后，再往后传。

通过 `RunnablePassthrough`，你可以在 `LangChain` 中以最小的成本实现数据增强，让流程保持简洁的同时具备灵活的扩展能力，尤其适合需要轻量级数据处理的场景（如日志、元数据添加、临时参数注入）

代码块

```
1  #RunnablePassthrough的使用
2  #RunnablePassthrough的两种用法都将在我们后面的课程中看到
3  from langchain_core.runnables import RunnableParallel, RunnablePassthrough
4
5  # RunnablePassthrough原样进行数据传递
6  runnable = RunnableParallel(
7      passed=RunnablePassthrough(),
8      modified=lambda x: x["num"] + 1,
9  )
10 print(runnable.invoke({"num": 1}))
11
12 # RunnablePassthrough对数据增强后传递
13 runnable = RunnableParallel(
14     passed=RunnablePassthrough().assign(query=lambda x: x["num"] + 2),
15     modified=lambda x: x["num"] + 1,
16 )
17 print(runnable.invoke({"num": 1}))
```

执行结果：

```
{'passed': {'num': 1}, 'modified': 2}
```

```
{'passed': {'num': 1, 'query': 3}, 'modified': 2}
```

RunnableWithMessageHistory

大模型是无状态的，也就是大模型是记不住我们聊天中每次的对话内容的。

代码块

```
1  #展示大模型的无状态，记不住我们聊天中每次的对话内容
2  from langchain_core.output_parsers import StrOutputParser
3  from langchain_core.prompts import ChatPromptTemplate,
   SystemMessagePromptTemplate
4
5  from models import get_lc_model_client
6  client = get_lc_model_client()
7
8  chat_template = ChatPromptTemplate.from_messages(
9      [
10         SystemMessagePromptTemplate.from_template("你是人工智能助手"),
11         ('human', '{text}')
12     ]
13 )
14 parser = StrOutputParser()
15
16 chain = chat_template | client | parser
17 print(chain.invoke({'text': '你好，我是云帆'}))
18 print(chain.invoke({'text': '你好，我是谁?'}))
```

为了让大模型能够记住我们聊天中每次的对话内容，需要额外进行处理。早期LangChain中有ConversationBufferMemory之类的记忆组件，现在这些组件已经被声明为废弃。

新版本的LangChain提供了专门的消息历史组件ChatMessageHistory和自动会话历史管理组件RunnableWithMessageHistory

ChatMessageHistory的使用

代码块

```
1  #消息历史组件ChatMessageHistory的使用
2  import os
3
4  from langchain_core.output_parsers import StrOutputParser
5  from langchain_core.prompts import ChatPromptTemplate,
   SystemMessagePromptTemplate, MessagesPlaceholder
6  from langchain_openai import ChatOpenAI
7  from langchain_community.chat_message_histories import ChatMessageHistory
8  from models import get_lc_model_client
```

```

9
10 chat_template = ChatPromptTemplate.from_messages(
11     [
12         SystemMessagePromptTemplate.from_template("你是人工智能助手"),
13         MessagesPlaceholder(variable_name="messages"),
14         #("placeholder", "{messages}"),
15     ]
16 )
17
18 client = get_lc_model_client()
19
20 parser = StrOutputParser()
21 chain = chat_template | client | parser
22
23 chat_history = ChatMessageHistory()
24
25 chat_history.add_user_message('你好，我是云帆') #用户提问加入历史记录
26 response = chain.invoke({'messages': chat_history.messages})
27 print(response)
28 chat_history.add_ai_message(response) #模型应答加入历史记录
29
30 print(chat_history.messages)
31 chat_history.add_user_message('你好，我是谁? ') #用户提问加入历史记录
32 print(chain.invoke({'messages': chat_history.messages}))

```

使用了ChatMessageHistory需要我们每次手动保存消息，有点不方便，所以这时可以使用RunnableWithMessageHistory

RunnableWithMessageHistory

接下来，我们完成一个可以支持多人同时和大模型对话的聊天机器人，同时支持流式输出

代码块

```

1  #自动会话历史管理组件RunnableWithMessageHistory
2  #以及如何流式处理大模型的应答
3  from langchain.messages import HumanMessage
4  from langchain_core.output_parsers import StrOutputParser
5  from langchain_core.prompts import ChatPromptTemplate,
6  SystemMessagePromptTemplate, MessagesPlaceholder
7  from langchain_core.runnables import RunnableWithMessageHistory
8  from langchain_community.chat_message_histories import ChatMessageHistory
9  from models import get_lc_model_client
10
11 client = get_lc_model_client()
12
13 #定义提示模版

```

```

13 prompt_template = ChatPromptTemplate.from_messages(
14     [
15         SystemMessagePromptTemplate.from_template("你是一个聊天助手，用{language}
回答所有的问题"),
16         #表示需要把以前的聊天记录作为对话内容的一部分发给大模型
17         MessagesPlaceholder(variable_name="history"),
18         ("human", "{input}"),
19     ]
20 )
21
22 parser = StrOutputParser()
23 # 以链的形式
24 chain = prompt_template | client | parser
25
26 #保存所有用户的聊天历史记录
27 store = {}
28
29 #接收用户的session_id，获取用户聊天记录
30 '''这里我们的是将用户的会话历史保存在本机内存中，在实际业务中一般会保存到Redis缓存，
31 代码一般如下所示，注意代码未经测试，只做示范：
32 REDIS_URL = os.getenv("REDIS_URL", "redis://localhost:6379")
33 def get_redis_history(session_id: str) -> BaseChatMessageHistory:
34     return RedisChatMessageHistory(session_id, redis_url=REDIS_URL)'''
35 def get_session(session_id:str):
36     if session_id not in store:
37         store[session_id] = ChatMessageHistory()
38     return store[session_id]
39
40 chatbot_with_his = RunnableWithMessageHistory(
41     chain,
42     get_session,
43     input_messages_key= "input",
44     history_messages_key= "history"
45 )
46
47 config_chn = {'configurable':{'session_id':'yunfang_chinese'}}
48 config_eng = {'configurable':{'session_id':'yunfang_english'}}
49
50 resp = chatbot_with_his.invoke(
51     {
52         "input": [HumanMessage(content="你好，我是云帆。")],
53         "language": "中文"
54     },
55     config = config_chn
56 )
57 print(resp)
58

```

```

59 resp = chatbot_with_his.invoke(
60     {
61         "input": [HumanMessage(content="你好，我是云帆。")],
62         "language": "英文"
63     },
64     config = config_eng
65 )
66 print(resp)
67
68 resp = chatbot_with_his.invoke(
69     {
70         "input": [HumanMessage(content="请问我的名字是什么? ")],
71         "language": "中文"
72     },
73     config = config_chn
74 )
75
76 print(resp)
77
78 resp = chatbot_with_his.invoke(
79     {
80         "input": [HumanMessage(content="请问我的名字是什么? ")],
81         "language": "英文"
82     },
83     config = config_eng
84 )
85 print(resp)
86
87 print(store)
88 #流式处理
89 # for respon in do_message.stream(
90 #     {
91 #         "msgs": [HumanMessage(content="请给我说一个有趣的笑话")],
92 #         "language": "中文"
93 #     },
94 #     config = config
95 # ):
96 #     print(respon.content,end="-")
97

```

常用LCEL链

!! LangChain v1.0已经把各种内置Chain移到 `langchain-classic` 中，以下内容简单了解即可！

LangChain已经事先做好了很LCEL链，可以到LangChain官方API查询：

https://python.langchain.com/api_reference/langchain/chains.html

但是注意：在“Deprecated classes”和“Deprecated functions”中的属于被声明废弃，不建议使用。

我们用create_stuff_documents_chain：文档链 看看如何使用。更多的LCEL链的使用，将在后面的课程中慢慢用到

create_stuff_documents_chain链将获取文档列表并将它们全部格式化为提示(文档列表)，然后将该提示传递给LLM。

代码块

```
1  from langchain_core.prompts import ChatPromptTemplate
2  from langchain_classic.chains.combine_documents import
   create_stuff_documents_chain
3  import bs4
4  from langchain_community.document_loaders import WebBaseLoader
5  from langchain_text_splitters import RecursiveCharacterTextSplitter
6
7  from models import get_lc_model_client
8
9  #获得访问大模型客户端
10 client = get_lc_model_client()
11
12 # 创建提示模板
13 prompt = ChatPromptTemplate.from_messages(
14     [
15         ("system", "根据提供的上下文: {context} \n\n 回答问题"),
16         ("user", "问题: {input}"), # 用户输入的问题
17     ]
18 )
19
20 # 构建链 这个链将文档作为输入，并使用之前定义的提示模板和初始化的大模型来生成答案
21 chain = create_stuff_documents_chain(client, prompt)
22
23 # 加载文档
24 loader = WebBaseLoader(
25     web_path="https://www.gov.cn/xinwen/2020-06/01/content_5516649.htm",
26     bs_kwargs=dict(parse_only=bs4.SoupStrainer(id="UCAP-CONTENT"))
27 )
28
29 docs = loader.load()
30
31 # 分割文档
32 text_splitter = RecursiveCharacterTextSplitter(chunk_size=1000,
   chunk_overlap=50)
```

```

33 documents = text_splitter.split_documents(docs)
34 # print(documents)
35 print(len(documents))
36
37 # 执行链 检索 民事法律行为? 出来的结果
38 res = chain.invoke({"input": "民事法律行为?", "context": documents[:5]})
39 print(res)

```

查看链及其组件

很多的时候，我们的业务中组成的链可能非常复杂，比如LangChain官网有这么一段话：

While we have seen users run chains with hundreds of steps in production, we generally recommend using LCEL for simpler orchestration tasks. When the application requires complex state management, branching, cycles or multiple agents, we recommend that users take advantage of [LangGraph](#).

（我们已经看到有人在生产中成功运行了包含 100 多个步骤的 LCEL 链）

这个时候，我们可能需要查看这个链上的组件都有哪些？执行顺序是怎么样的？各个组件接收什么样的输入，产生什么样的输出？

代码块

```

1  #查看链及其组件
2  import os
3
4  from langchain.messages import SystemMessage, HumanMessage
5  from langchain_core.output_parsers import StrOutputParser
6  from langchain_core.prompts import ChatPromptTemplate,
   SystemMessagePromptTemplate, HumanMessagePromptTemplate, \
7      PromptTemplate
8  from langchain_core.runnables import RunnableSequence
9  from langchain_openai import ChatOpenAI
10
11 from models import get_lc_model_client
12
13 #获得访问大模型客户端
14 client = get_lc_model_client()
15
16 #使用ChatPromptTemplate，它是个大模版，可以组合各种角色的消息模板
17 #定义提示模版
18 '''LangChain提供不同类型的角色的消息模板。
19 最常用的是AIMessagePromptTemplate、SystemMessagePromptTemplate和
   HumanMessagePromptTemplate，
20 分别创建代表人工智能应答消息、系统消息（提示词中的角色、任务、参考案例、约束项等等）和人类
   消息（具体的任务）。'''
21 chat_template = ChatPromptTemplate.from_messages(

```

```

22     [
23         #改为 ('system', '请将以下的内容翻译成{language}') 也可以
24         SystemMessagePromptTemplate.from_template("请将以下的内容翻译成
{language}"),
25         # 改为 HumanMessagePromptTemplate.from_template("{text}") 也可以
26         ('human', '{text}')]
27     ]
28 )
29
30 # #2、解析返回结果
31 parser = StrOutputParser()
32
33 chain = chat_template | client | parser
34 print(parser.input_schema.model_json_schema())
35 print(parser.output_schema.model_json_schema())
36 print(chain.input_schema.model_json_schema())
37 print(chain.output_schema.model_json_schema())
38 print(chain.get_prompts())
39 #打印出链的结构图，该功能需要额外安装 pip install grandalf
40 chain.get_graph().print_ascii()
41 # print(chain.invoke({'language': '意大利文', 'text': '朋友啊再见!'}))

```

LangChain与RAG

这个部分我们将会了解到 数据连接（Data connection）这个核心组件

在许多LLM应用程序中，用户特定的数据不在大模型中，可能在外部系统或文档中。如何使用这些外部数据来增强呢？

LangChain中的数据连接组件包括几个关键模块：文档加载器、文档切分、文本嵌入、向量存储、检索器 等等。

分两步了解：

- 1、文档加载、切割、转换、嵌入、向量存储和检索与大模型生成，完成一个基础RAG应用
- 2、基于在线网页的RAG实现

文档与文档加载

文档类型有很多，可以是简单的文本文件、Word文档、Excel表格、Pdf文件等等，甚至是 各种视频的转录文件

加载器有 load方法，用于从指定的数据源读取数据，并将其转换成一个或多个文档。这使得 LangChain 能够处理各种形式的输入数据，不仅仅限于文本数据，还可以是网页、视频字幕等。

LangChain有很强的数据加载能力，提供了很多常见的数据格式的支持，例如CSV、文件目录、HTML、JSON、Markdown及PDF等。

前面的对create_stuff_documents_chain（文档链）使用中已经看到了加载器的使用。

在前面的RAG基础章节，我们已经了解了文本的分割。LangChain为此也提供了很多现成的分割器：

CharacterTextSplitter： 基于字符(默认为"\n\n")进行切割，并通过字符数量来测量文本块的大小。使用chunk_size属性可设置文本块的大小，使用chunk_overlap 属性设置文本块之间的最大重叠。

RecursiveCharacterTextSplitter： 在前面我们已经了解过，除了自然语言，它也支持对特定的编程语言(‘JavaScript’、‘cpp’、‘go’、‘java’、‘php’、‘python’等等)的代码进行切割。

MarkdownHeaderTextSplitter： 可以根据指定的一组标题来切割一个Markdown 文档。

.....

内置分割器：https://python.langchain.com/api_reference/text_splitters/index.html

LangChain文档转换器处理任务分为两个步骤：

第一步是对文档进行切割，主要由切割器完成；

第二步是将切割后的文档转换为 Document 对象/数据格式。

LangChain中数据以 Document 对象和向量形式在各个包装器中流通。Document 对象包含文本及其相关元数据，它是LangChain在RAG处理中非常重要的概念。

文档加载器的主要任务是从各种源加载数据转换为 Document 对象，文档转换器将文档数据切割也转换为Document对象。

这些 Document 对象会被传递给嵌入模型包装器，嵌入模型包装器再将它们转换为嵌入向量，被存储在向量存储库中。

代码块

```
1  #文档的模拟数据
2  documents = [
3      Document(
4          page_content="猫是柔软可爱的动物，但相对独立",
5          metadata={"source": "常见动物宠物文档"},
6      ),
7      Document(
8          page_content="狗是人类很早开始的动物伴侣，具有团队能力",
9          metadata={"source": "常见动物宠物文档"},
10     ),
11     Document(
12         page_content="金鱼是我们常常喂养的观赏动物之一，活泼灵动",
13         metadata={"source": "鱼类宠物文档"},
14     ),
15     Document(
16         page_content="鹦鹉是猛禽，但能够模仿人类的语言",
17         metadata={"source": "飞禽宠物文档"},
18     ),
19     Document(
```

```
20         page_content="兔子是小朋友比较喜欢的宠物，但是比较难喂养",
21         metadata={"source": "常见动物宠物文档"},
22     ),
23 ]
```

文本嵌入模型

文本嵌入模型（Embedding Model），这些模型将文本作为输入并返回词向量。

LangChain 框架提供了一个名为Embeddings 的类，它为多种文本嵌入模型(如OpenAI、Cohcre、HuggingFace 等)提供了统一的接口。通过该类实例化的嵌入模型包装器，可以将文档转换为向量数据，同时将搜索的问题也转换为向量数据，这使得可通过计算搜索问题和文档在向量空间中的距离，来寻找在向量空间中最相似的文本。

实例化的 Embeddings 类被称为嵌入模型包装器，同Model /O 模块的 LLM 模型包装器和聊天模型包装器(Chat Model)并称为三大模型包装器。

代码块

```
1  # 获得一个阿里通义千问嵌入模型的实例
2  from langchain_community.embeddings import DashScopeEmbeddings
3  llm_embeddings = DashScopeEmbeddings(
4      model=ALI_TONGYI_EMBEDDING_MODEL,
5      dashscope_api_key=os.getenv(ALI_TONGYI_API_KEY_SYSVAR_NAME)
6  )
```

LangChain 提供了很多嵌入模型的实现，可以查询langchain_community\embeddings__init__.py 获得具体的列表。

文本向量化以后，需要保存起来，以供后面的步骤使用，就需要向量数据库。

向量数据库

在前面的RAG基础章节，我们已经了解了向量数据库，这里不再详述。接下来，直接看看LangChain 框架中如何使用文本嵌入模型和向量数据库。

代码块

```
1  # 实例化向量空间
2  vector_store =
3      Chroma.from_documents(documents=documents, embedding=llm_embeddings)
4
5  #相似度查询
6  print(vector_store.similarity_search("狸花猫"))
7  print("--"*15)
8  #按相似度的分数进行排序，分数值越小，越相似
```

```
8 print(vector_store.similarity_search_with_score("狸花猫"))
9 #检索器, bind(k=1)表示返回相似度最高的第一个
10 docs_find = RunnableLambda(vector_store.similarity_search).bind(k=1)
11 print(docs_find.batch(["狸花猫", "海豚"]))
```

加入大模型进行提问

代码块

```
1 import os
2
3 import langchain
4 from langchain_chroma import Chroma
5 from langchain_core.prompts import ChatPromptTemplate
6 from langchain_core.runnables import RunnableLambda, RunnablePassthrough
7 from langchain_core.documents import Document
8
9 from models import get_lc_model_client, ALI_TONGYI_API_KEY_OS_VAR_NAME,
    ALI_TONGYI_EMBEDDING_MODEL, get_ali_model_client
10
11 # langchain.debug = True
12
13 #获得访问大模型客户端
14 client = get_ali_model_client()
15
16 #直接了解LangChain中的“文档”(Document)的具体内容, 这里我们跳过了文档与文档加载, 文档切
    割和文档转换过程
17 #文档的模拟数据
18 documents = [
19     Document(
20         page_content="猫是柔软可爱的动物, 但相对独立",
21         metadata={"source": "常见动物宠物文档"},
22     ),
23     Document(
24         page_content="狗是人类很早开始的动物伴侣, 具有团队能力",
25         metadata={"source": "常见动物宠物文档"},
26     ),
27     Document(
28         page_content="金鱼是我们常常喂养的观赏动物之一, 活泼灵动",
29         metadata={"source": "鱼类宠物文档"},
30     ),
31     Document(
32         page_content="鹦鹉是猛禽, 但能够模仿人类的语言",
33         metadata={"source": "飞禽宠物文档"},
34     ),
35     Document(
```

```

36         page_content="兔子是小朋友比较喜欢的宠物，但是比较难喂养",
37         metadata={"source": "常见动物宠物文档"},
38     ),
39 ]
40
41 # 获得一个阿里通义千问嵌入模型的实例，同样在models.py中被包装为get_alibaba_embeddings()
42 from langchain_community.embeddings import DashScopeEmbeddings
43 llm_embeddings = DashScopeEmbeddings(
44     model=ALI_TONGYI_EMBEDDING_MODEL,
45     dashscope_api_key=os.getenv(ALI_TONGYI_API_KEY_OS_VAR_NAME)
46 )
47 # 实例化向量空间
48 vector_store =
49     Chroma.from_documents(documents=documents, embedding=llm_embeddings)
50
51 #展示相似度查询，实际业务中可以不要
52 print(vector_store.similarity_search("狸花猫"))
53 print("---"*15)
54 #按相似度的分数进行排序，分数值越小，越相似（其实是L2距离）
55 print(vector_store.similarity_search_with_score("狸花猫"))
56
57 #检索器，bind(k=1)表示返回相似度最高的第一个
58 docs_find = RunnableLambda(vector_store.similarity_search).bind(k=1)
59 # print(docs_find.batch(["狸花猫", "海豚"]))
60
61 message = """
62 仅使用提供的上下文回答下面的问题：
63 {question}
64 上下文：
65 {context}
66 """
67 prompt_template = ChatPromptTemplate.from_messages([('human', message)])
68 # 定义这个链的时候，还不知道问题是什么，
69 # 用RunnablePassthrough允许我们将用户的具体问题在实际使用过程中进行动态传入
70 # chain = {"question":RunnablePassthrough(),"context":docs_find} |
71     prompt_template | client
72 # #用大模型生成答案
73 # resp = chain.invoke("请介绍一下猫")
74 # print(resp.content)
75
76 questiong = "请介绍一下猫"
77 # '''如果不用RunnablePassthrough(), 那么需要这么写，已经测试通过，直接使用docs_find不行'''
78 # chain1 = prompt_template | client

```

```

78 # resp = chain1.invoke({'question': questiong,
    "context":vector_store.similarity_search(questiong,k=1)})
79 # resp = chain1.invoke({'question': questiong, "context":docs_find})
80 # print(resp.content)

```

看起来，检索器可有可无，因为向量库本身就提供了检索功能，为何LangChain中还要提供一个检索器呢？

检索器

向量存储库种类繁多，比如Chroma、FAISS、Pinecone、等。若直接和这些向量存储库进行交互，则可能需要具备深入的数据库操作知识，如了解查询语法，管理数据库连接，处理错误和异常等。这样的操作可能较为复杂，并带来不便。

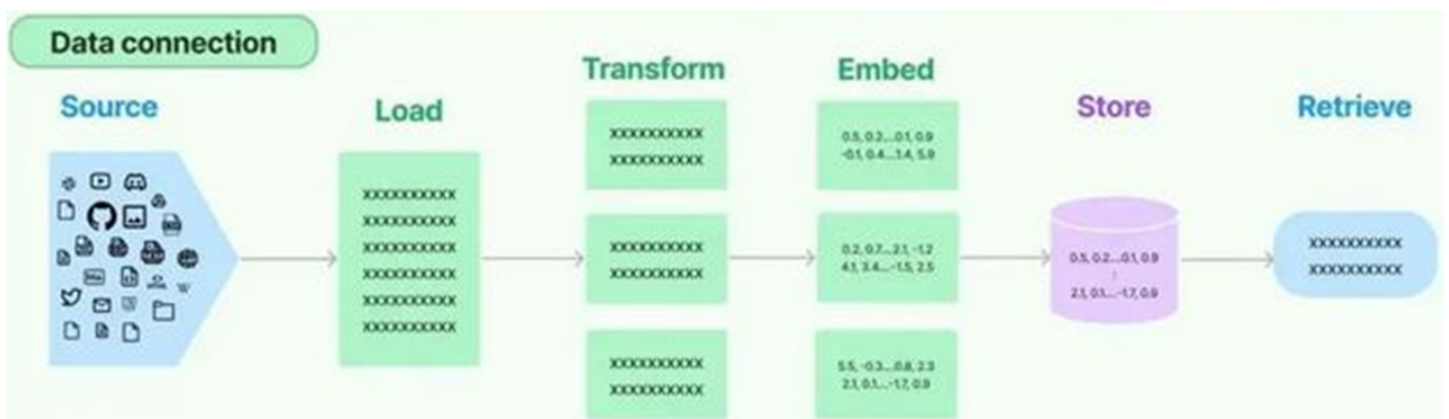
如果有一种方法，能将各种向量存储库统一到一个接口上，那就非常方便了。LangChain 为我们做了这个事情，它封装了 VectorStoreRetriever 类，提供了一个标准接口。

可以通过在向量存储库的实例上调用as_retriever 方法得到一个基于向量存储库的检索器，即 VectorStoreRetriever 类的实例。

所以可以把检索器看作一个向量存储库的包装器，它包装了统一的接口，无论底层的向量存储库是什么，都可以使用同样的方式对其进行查询，可以轻松地切换不同的向量存储库，而无须修改查询代码。

完成基于LangChain的在线网页的RAG实现

前面的组件串起来，可形成如下的流程：



代码块

```

1 import os
2 import bs4
3 from langchain.chains.combine_documents import create_stuff_documents_chain
4 from langchain.chains.retrieval import create_retrieval_chain
5 from langchain_community.document_loaders import WebBaseLoader
6 from langchain_community.vectorstores import Chroma

```

```

7  from langchain_core.prompts import ChatPromptTemplate
8  from langchain_text_splitters import RecursiveCharacterTextSplitter
9
10 from models import get_lc_model_client, ALI_TONGYI_EMBEDDING_MODEL,
    ALI_TONGYI_API_KEY_SYSVAR_NAME
11
12 #获得访问大模型客户端
13 client = get_lc_model_client()
14
15 # 获得一个嵌入模型的实例
16 from langchain_community.embeddings import DashScopeEmbeddings
17 llm_embeddings = DashScopeEmbeddings(
18     model=ALI_TONGYI_EMBEDDING_MODEL,
19     dashscope_api_key=os.getenv(ALI_TONGYI_API_KEY_SYSVAR_NAME)
20 )
21
22 #获得文档，文档内容来自网页
23 loader = WebBaseLoader(
24     web_path=
25     ["https://www.news.cn/fortune/20250212/895ac6738b7b477db8d7f36c315aae22/c.html"
26     ],
27     bs_kwargs=dict(
28         parse_only = bs4.SoupStrainer(class_=("main-left left","title"))
29     )
30 )
31 docs = loader.load()
32 # print(len(docs))
33 # print(docs)
34
35 #文本的切割
36 splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=100)
37 documents = splitter.split_documents(docs)
38 # for s in documents:
39 #     print(s,end="**\n")
40
41 # 实例化向量空间
42 vector_store =
43 Chroma.from_documents(documents=documents,embedding=llm_embeddings)
44
45 #检索器
46 retriever = vector_store.as_retriever()
47
48 #整合
49 system_prompt = """
50 您是问答任务的助理。使用以下检索到的上下文来回答问题。
51 如果你不知道答案，就说你不知道。
52 """

```

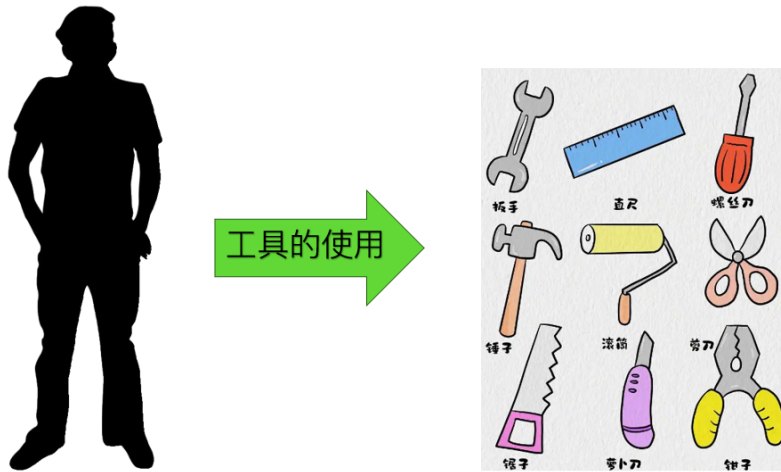
```

49 prompt_template = ChatPromptTemplate.from_messages(
50     [
51         ("system", system_prompt),
52         ("assistant", "{context}"),
53         ("human", "{input}")
54     ]
55 )
56
57 #创建链
58 chain1 = create_stuff_documents_chain(client,prompt_template)
59 chain2 = create_retrieval_chain(retriever,chain1)
60 # 用大模型生成答案
61 resp = chain2.invoke({"input":"张成刚是谁? "})
62 #print(resp)
63 print(resp["answer"])

```

LangChain的工具调用（已更新）

如果大模型只是和我们进行自然语言聊天，它固然有用，但是决定没有现在这样用途广泛。我们希望大模型能做更多的事，比如像人类一样使用工具



现在比较火爆的MCP、Agent2Agent说到底，都离不开大模型对工具的调用，那么大模型是如何进行工具调用的呢？

代码块

```

1 import gradio as gr
2 import os
3 import subprocess
4 import webbrowser
5 from datetime import datetime
6 from langchain.agents import create_agent
7 from langchain.messages import SystemMessage
8 from langchain.tools import tool

```

```
9  from langchain_openai import ChatOpenAI
10
11  # 设置环境变量
12  DASHSCOPE_API_KEY = os.getenv("DASHSCOPE_API_KEY")
13
14  # 初始化消息历史
15  INIT_MESSAGE = SystemMessage(content="你通义千问提供的人工智能助手，你是百科全书。")
16
17  # 定义工具函数
18  @tool
19  def get_current_time(input: str = "") -> str:
20      """获取当前时间"""
21      current_datetime = datetime.now()
22      formatted_time = current_datetime.strftime('%Y-%m-%d %H:%M:%S')
23      result = f"当前时间: {formatted_time}。"
24      print(result)
25      return result
26
27  @tool
28  def recom_drink(input: str = "") -> str:
29      """推荐附近的饮品店"""
30      result = '''距离您500米内有如下饮料店: \n
31      1、蜜雪冰城\n
32      2、茶颜悦色\n
33      另外距离您200米内有惠民便利店，里面应该有矿泉水或其他饮品'''
34      return result
35
36  @tool
37  def open_calc(input: str = "") -> str:
38      """打开计算器"""
39      try:
40          subprocess.Popen(['calc.exe'])
41          return "计算器已打开"
42      except Exception as e:
43          return f"打开计算器失败: {str(e)}"
44
45  @tool
46  def open_browser(url: str) -> str:
47      """打开浏览器访问指定网址"""
48      try:
49          webbrowser.open(url)
50          return f"已打开浏览器访问 {url}"
51      except Exception as e:
52          return f"打开浏览器失败: {str(e)}"
53
54  # 创建LangChain工具
```



```

55     tools = [
56         get_current_time,
57         recom_drink,
58         open_calc,
59         open_browser
60     ]
61
62     # 初始化聊天模型和记忆
63     llm = ChatOpenAI(
64         openai_api_base="https://dashscope.aliyuncs.com/compatible-mode/v1",
65         openai_api_key=DASHSCOPE_API_KEY,
66         model_name="qwen-max"
67     )
68
69     # 创建记忆
70     # 短期记忆构建：使用InMemorySaver()实现单会话的短期记忆
71     from langgraph.checkpoint.memory import InMemorySaver
72     # 创建短期记忆实例
73     memory = InMemorySaver()
74
75     # 创建提示模板
76     system_prompt = "你是人工智能助手。需要帮助用户解决各种问题。"
77     # 这个时候函数工具选择和调用执行，都是通过Agent来完成的
78     # 创建Agent
79     agent = create_agent(
80         model=llm,
81         tools=tools,
82         system_prompt=system_prompt,
83         checkpointinter=memory # 传入记忆组件
84     )
85
86     # agent = initialize_agent(
87     #     tools, # 工具列表
88     #     llm, # 聊天模型
89     #     agent=AgentType.OPENAI_FUNCTIONS,
90     #     verbose=True,
91     #     memory=memory, # 记忆
92     #     agent_kwargs=agent_kwargs,
93     #     handle_parsing_errors=True
94     # )
95
96     # 处理LLM响应
97     def process_llm_response(query, show_history):
98         if len(query) == 0:
99             return show_history + [("", "")]
100         try:
101             # 显示用户查询和等待提示

```

```

102     yield show_history + [(query, "正在查询大模型...")], ""
103
104     # 使用Agent处理查询
105     response = agent.invoke(
106         {"messages": [{"role": "user", "content": query}]},
107         # 配置会话标识, 用于区分不同用户
108         config={"configurable": {"thread_id": "user_1"}} # 会话唯一标识, 用
于区分不同用户
109     )
110     print(f"LLM输出: {response}")
111
112     # 正确提取回答内容 - 只显示output部分
113     if isinstance(response, dict) and 'messages' in response:
114         response = response["messages"][-1].content
115     else:
116         # 如果response不是预期的字典格式, 尝试提取其他可能的字段
117         response = str(response)
118         print(f"警告: 响应格式异常: {response}")
119
120     # 返回结果
121     yield show_history + [(query, response)], ""
122 except Exception as e:
123     print(f"Error: {e}")
124     yield show_history + [(query, "AI助手出错, 请重试或者检查")]
125
126 # 前端界面展示
127 with gr.Blocks(title="大模型中Function Call演示") as demo:
128     gr.HTML('<center><h1>欢迎大模型中Function Call演示</h1></center>')
129
130     with gr.Row():
131         with gr.Column(scale=10):
132             chatbot = gr.Chatbot(height=650) # 空初始值
133
134     with gr.Row():
135         msg = gr.Textbox(label="输入", placeholder="您想了解什么呢? ")
136
137     with gr.Row():
138         examples = gr.Examples(
139             examples=[
140                 '请问如何做红烧牛肉? ',
141                 '料酒可以换成白酒吗? ',
142                 '帮我打开计算器',
143                 '现在几点了? ',
144                 '帮我访问淘宝网',
145                 '我渴了'
146             ],
147             inputs=[msg]

```

```

148         )
149
150         clear = gr.ClearButton([chatbot, msg])
151         msg.submit(process_llm_response, [msg, chatbot], [chatbot,msg])
152
153     if __name__ == '__main__':
154         demo.launch(server_port=7778)

```

工具Tools

工具是代理、链或LLM可以用来与世界互动的接口。它们结合了几个要素

- 工具的名称
- 工具的描述
- 该工具输入的JSON模式
- 要调用的函数
- 是否应将工具结果直接返回给用户

LangChain通过提供统一框架集成功能的具体实现，在框架内，每个功能被封装成一个工具。Tools 组件中可以调用的各种工具类。LangChain 提供了一组预定义的Tools 组件，同时也允许用户自定义Tools组件以满足特定的需求。

Toolkits 组件是一个特殊的组件集合，它包括了多个用于实现特定目标的 Tools组件，LangChain 也提供了预定义的 Toolkits 组件。

实战：使用工具查询数据库表

1、LangChain访问MySQL数据库

代码块

```

1  #LangChain访问MySQL数据库
2  from langchain_community.utilities import SQLDatabase
3
4  #数据库配置
5  HOSTNAME = '127.0.0.1'
6  PORT = '3306'
7  DATABASE = 'world'
8  USERNAME = 'root'
9  PASSWORD = '123456'
10 MySQL_URI = 'mysql+mysqldb://{}:{ }@{ }:{ }/{ }?
    charset=utf8mb4'.format(USERNAME, PASSWORD, HOSTNAME, PORT, DATABASE)
11 db = SQLDatabase.from_uri(MYSQL_URI)
12 print(db.get_usable_table_names())
13 print(db.run('select * from country limit 1'))

```

2、将工具和大模型进行绑定

代码块

```
1  '''学习如何将工具和大模型进行绑定和如何自定义工具
2  需要安装
3  pip install mysqlclient
4  pip install SQLAlchemy'''
5  # 添加PyMySQL适配代码
6  import pymysql
7  pymysql.install_as_MySQLdb()
8
9  import os
10 from langchain.messages import HumanMessage
11 from langchain_community.utilities import SQLDatabase
12 from langchain.tools import tool
13 from langchain_community.tools import TavilySearchResults
14 from models import get_lc_model_client
15
16 #获得访问大模型客户端，这里为什么要改变模型？
17 client = get_lc_model_client()
18
19 #数据库配置
20 HOSTNAME = '127.0.0.1'
21 PORT = '3306'
22 DATABASE = 'world'
23 USERNAME = 'root'
24 PASSWORD = 'root123456'
25 MYSQL_URI = 'mysql+mysql://{}:{}_{}@{}:{}_{}?
charset=utf8mb4'.format(USERNAME,PASSWORD,HOSTNAME,PORT,DATABASE)
26 db = SQLDatabase.from_uri(MYSQL_URI)
27
28 # 定义一个函数，用于获取数据库中的所有表名，并且包装为LangChain中的工具
29 @tool
30 def get_table_names():
31     """获取数据库中的所有表名"""
32     return db.get_table_names()
33
34 # 把工具绑定到大模型中
35 client_with_tools = client.bind_tools([get_table_names])
36
37 resp = client_with_tools.invoke([HumanMessage(content="请从国家表中查询出China的所有数据")])
38 print(resp)
39 print("**",resp.content)
```

```

40 print("**",resp.tool_calls)
41 '''
42 从上面的执行结果可以看到，大模型判断出为了回答问题，需要使用工具，但是工具的使用不是大模型
    负责的，而应该是我们的应用负责的，
43 所以在在大模型使用工具回答问题的过程中，往往需要多次和大模型交互才能得到最终的结果。
44 '''
45

```

3、使用内置工具

代码块

```

1  '''下面展示如何使用LangChain的内置工具，比如搜索引擎TavilySearchResults
2  注意该工具需要科学上网，以及申请API_KEY才能使用，所以看看就行了。https://tavily.com/
3  这段代码关注的重点：自定义工具和内置工具在使用上并无本质差别'''
4  search = TavilySearchResults(max_results=2)
5  client_with_tools = client.bind_tools([search])
6
7  result = client_with_tools.invoke([HumanMessage(content = "长沙的天气如何? ")])
8  print("==",result.content)
9  print("==",result.tool_calls)

```

4、LangChain查询数据库表

Langchain v0.3代码如下：

代码块

```

1  import os
2  from operator import itemgetter
3
4  import langchain
5  from langchain.chains.sql_database.query import create_sql_query_chain
6  from langchain_community.tools import QuerySQLDatabaseTool
7  from langchain_community.utilities import SQLDatabase
8
9  from langchain_core.output_parsers import StrOutputParser
10 from langchain_core.prompts import PromptTemplate
11 from langchain_core.runnables import RunnablePassthrough
12 from langchain_openai import ChatOpenAI
13 import re
14 from models import get_lc_model_client, get_ali_model_client,
    ALI_TONGYI_MAX_MODEL
15
16 #获得访问大模型客户端
17 #client = get_lc_model_client()

```

```

18 client = get_ali_model_client(model=ALI_TONGYI_MAX_MODEL)
19
20 #数据库配置
21 HOSTNAME = '127.0.0.1'
22 PORT = '3306'
23 DATABASE = 'world'
24 USERNAME = 'root'
25 PASSWORD = 'root123456'
26 MYSQL_URI = 'mysql+mysqldb://{username}:{password}@{hostname}:{port}/{database}?
charset=utf8mb4'.format(USERNAME, PASSWORD, HOSTNAME, PORT, DATABASE)
27 db = SQLiteDatabase.from_uri(MYSQL_URI)
28
29 '''对于问题："请从国家表中查询出China的所有数据"，要分为几步才能出结果：
30 1、大模型判断这个问题需要调用工具查询数据库，获得所有的表名和表中的字段名，
31 目的是看那个表才是国家表，国家表有哪些字段，
32 2、工具执行后，把工具执行结果交给大模型
33 3、大模型根据国家表和国家表中的字段，生成SQL语句
34 4、SQL语句的执行依然需要使用工具
35 5、工具执行后，把工具执行结果交给大模型，大模型生成最终答案'''
36
37 #1、用LangChain内置链create_sql_query_chain将大模型和数据库结合，会产生sql而不会执行
sql
38 sql_make_chain = create_sql_query_chain(client, db)
39 # resp = sql_make_chain.invoke({"question": "请从国家表中查询出China的所有数据"})
40 # print(resp)
41 # print("***15)
42
43 #2、因为实际产生的sql是形如``sql....``的，无法直接执行，所以需要清理
44 class SQLCleaner(StrOutputParser):
45     def parse(self, text: str) -> str:
46         pattern = r'``sql(?:.*?)``'
47         match = re.search(pattern, text, re.DOTALL)
48         if match:
49             sql = match.group(1).strip()
50             # 去除 SQLQuery: 前缀
51             sql = re.sub(r'^SQLQuery:', '', sql).strip()
52             return sql
53         # 去除 SQLQuery: 前缀
54         text = re.sub(r'^SQLQuery:', '', text).strip()
55         return text
56 sql_make_chain = create_sql_query_chain(client, db) | SQLCleaner()
57 # resp = sql_make_chain.invoke({"question": "请从国家表中查询出China的相关数据"})
58 # print("实际可用SQL: ", resp)
59 # print("***15)
60
61 #3、将前面的部分组合起来，得到最终结果
62 answer_prompt = PromptTemplate.from_template(

```

```

63     """给定以下用户问题、可能的SQL语句和SQL执行后的结果，回答用户问题
64     Question: {question}
65     SQL Query: {query}
66     SQL Result:{result}
67     回答: """
68 )
69 #创建一个执行SQL的工具
70 execute_sql_tools = QuerySQLDatabaseTool(db = db)
71 # runnable = RunnablePassthrough.assign(query=sql_make_chain)
72 # print(runnable.invoke({"question": "请从国家表中查询出China的相关数据"}))
73 # runnable = RunnablePassthrough.assign(query=sql_make_chain) |
74 #     itemgetter('query')
75 # print(runnable.invoke({"question": "请从国家表中查询出China的相关数据"}))
76 # runnable = RunnablePassthrough.assign(query=sql_make_chain) |
77 #     itemgetter('query') | execute_sql_tools
78 # print(runnable.invoke({"question": "请从国家表中查询出China的相关数据"}))
79 '''通过上面的步骤，就能搞清楚{question}、{query}、{result}这三个字段是如何通过LCEL链一步步获得的
80 要注意的是result=itemgetter('query')|execute_sql_tools 中，执行顺序是：
81 itemgetter('query') -> execute_sql_tools -> result=
82 所以这段代码实际是：result=(itemgetter('query')|execute_sql_tools)'''
83 chain =
84     (RunnablePassthrough.assign(query=sql_make_chain).assign(result=itemgetter('query')|execute_sql_tools)
85     | answer_prompt | client | StrOutputParser())
86 result = chain.invoke(input={"question": "请从国家表中查询出China的相关数据"})
87 #result = chain.invoke(input={"question": "请问国家表中有多少条数据"})
88 print("最终执行的结果：", result)

```

对于Langchain v1.0，create_agent取代了过去的 AgentExecutor 和各种 Chain。代码也可以这样写：

代码块

```

1 # 添加PyMySQL适配代码
2 import pymysql
3 pymysql.install_as_MySQLdb()
4
5 from langchain.agents import create_agent
6 from langchain_community.agent_toolkits import SQLDatabaseToolkit
7 from langchain_community.utilities import SQLDatabase
8 from langchain.messages import AIMessage

```

```

9  from models import get_ali_model_client, ALI_TONGYI_MAX_MODEL
10
11  client = get_ali_model_client(model=ALI_TONGYI_MAX_MODEL)
12
13  HOSTNAME = '127.0.0.1'
14  PORT = '3306'
15  DATABASE = 'world'
16  USERNAME = 'root'
17  PASSWORD = 'root123456'
18  MYSQL_URI = 'mysql+mysqldb://{host}:{port}@{user}:{password}/{database}?charset=utf8mb4'.format(USERNAME,
19  PASSWORD, HOSTNAME, PORT, DATABASE)
20  db = SQLiteDatabase.from_uri(MYSQL_URI)
21
22  # 1. 准备工具箱
23  toolkit = SQLiteDatabaseToolkit(db=db, llm=client)
24  tools = toolkit.get_tools()
25
26  # 2. 定义系统提示词
27  system_prompt = """你是一个精通MySQL数据库的AI助手。
28  你的任务是查询数据库并回答用户问题。
29
30  【极其重要的规则】
31  1. **绝对不要猜测表名**。你必须先使用工具查看数据库里到底有哪些表。
32  2. **绝对不要猜测字段名**。确定表名后，必须使用工具查看表的 Schema（结构）。
33  3. 只有在确认了表名和字段名之后，才能编写 SQL 语句。
34  4. 如果 SQL 执行报错，仔细查看错误信息，修正后重试。
35
36  注意：
37  - 永远不要执行 DROP, DELETE, INSERT, UPDATE 等修改数据的操作。
38  - 只输出最终的自然语言回答，不要暴露内部的 SQL 逻辑，除非用户要求。
39  """
40
41  # 3. 创建 Agent
42  # - 它内置了 Tool Calling 循环、错误重试和消息历史管理
43  agent = create_agent(
44      model=client,
45      tools=tools,
46      system_prompt=system_prompt
47  )
48
49  # --- 执行查询 ---
50  print("="*20 + " Agent 开始执行 " + "="*20)
51
52  question = "请从国家表中查询出China的相关数据"
53  # question = "请问国家表中有多少条数据"
54
55  # 4. 调用 Agent

```



```

55 # v1.0 Agent 接收标准的 messages 列表作为输入
56 # result 包含了完整的执行状态, 包括中间步骤
57 result = agent.invoke({"messages": [("user", question)]})
58 print(result)
59
60 # 5. 从历史消息中提取 SQL
61 print("\n" + "="*20 + " 提取实际执行的 SQL " + "="*20)
62
63 executed_sql = None
64 # 遍历消息历史 (顺序: User -> AI(Call Tool) -> Tool(Result) -> AI(Final Answer))
65 for msg in result['messages']:
66     # 只有 AIMessage 才包含工具调用请求
67     if isinstance(msg, AIMessage) and msg.tool_calls:
68         for tool_call in msg.tool_calls:
69             # 检查是否是执行SQL的工具 (SQLDatabaseToolkit 中通常叫 'sql_db_query')
70             if tool_call['name'] == 'sql_db_query':
71                 # 提取参数中的 query 字段
72                 executed_sql = tool_call['args'].get('query')
73
74 if executed_sql:
75     print(f"\n✅ 最终使用的 SQL 语句:\n{executed_sql}")
76 else:
77     print("\n❌ 未检测到 SQL 执行")
78
79 # 6. 打印最终自然语言回答
80 print("\n" + "="*20 + " 最终自然语言回答 " + "="*20)
81 print(result["messages"][-1].content)

```

```
===== Agent 开始执行 =====  
  
===== 提取实际执行的 SQL =====  
  
✅ 最终使用的 SQL 语句：  
SELECT * FROM country WHERE Name = 'China'  
  
===== 最终自然语言回答 =====  
查询结果显示，以下是与中国（China）相关的数据：  
  
- **国家代码（Code）**：CHN  
- **国家名称（Name）**：China  
- **所在洲（Continent）**：Asia  
- **所属区域（Region）**：Eastern Asia  
- **面积（SurfaceArea）**：9,572,900.00 平方公里  
- **独立年份（IndepYear）**：-1523（可能表示公元前1523年）  
- **人口数量（Population）**：1,277,558,000  
- **平均寿命（LifeExpectancy）**：71.4 岁  
- **国民生产总值（GNP）**：982,268.00  
- **旧的国民生产总值（GNPold）**：917,719.00  
- **本地名称（LocalName）**：Zhongquo  
- **政府形式（GovernmentForm）**：People's Republic  
- **国家元首（HeadOfState）**：Jiang Zemin  
- **首都ID（Capital）**：1891  
- **另一个国家代码（Code2）**：CN  
  
如果您需要进一步的信息或有其他问题，请随时告诉我！
```

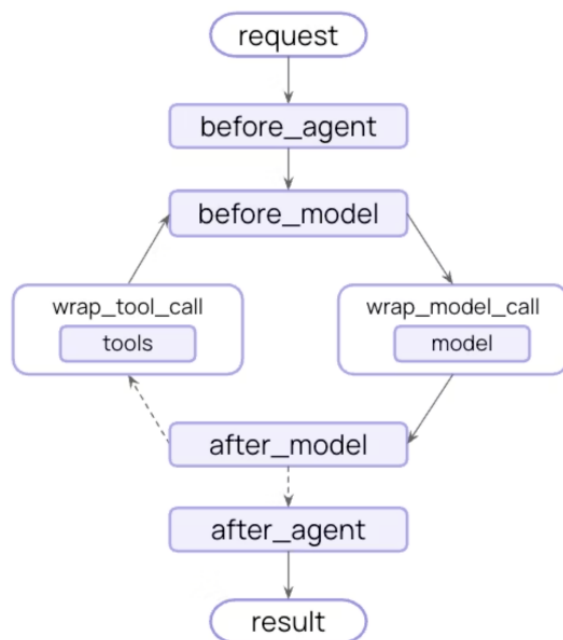
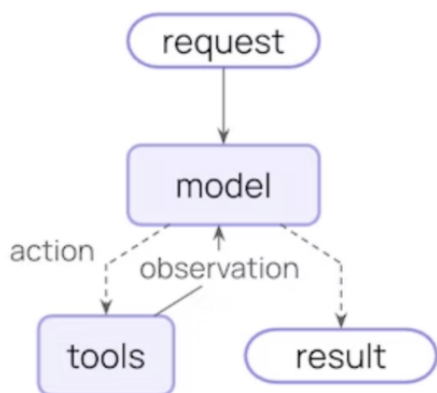
中间件（Middleware）（已更新）

!! 中间件通常搭配 Agent 来用，在本阶段简单了解即可！

什么是中间件？

中间件（Middleware）是一种流程控制机制，用于在智能体执行过程中拦截、修改或增强请求与响应的处理逻辑，而无需修改核心 Agent 或工具的代码。这是 v1.0 最大的亮点之一，可以像在 Web 开发中一样，在 Agent 的生命周期中插入自定义逻辑。

中间件在每个步骤之前和之后都会暴露钩子（函数）



中间件能做什么？



监视器

通过日志记录、分析和调试来跟踪代理行为



调整

转换提示、工具选择和输出格式



控制

添加重试、回退和提前终止逻辑



执行

应用速率限制、防护措施和个人身份信息检测

内置中间件

参考链接：<https://docs.langchain.com/oss/python/langchain/middleware/built-in>

Middleware	Description
<u>Summarization</u>	Automatically summarize conversation history when approaching token limits.
<u>Human-in-the-loop</u>	Pause execution for human approval of tool calls.
<u>Model call limit</u>	Limit the number of model calls to prevent excessive costs.
<u>Tool call limit</u>	Control tool execution by limiting call counts.
<u>Model fallback</u>	Automatically fallback to alternative models when primary fails.
<u>PII detection</u>	Detect and handle Personally Identifiable Information (PII).
<u>To-do list</u>	Equip agents with task planning and tracking capabilities.
<u>LLM tool selector</u>	Use an LLM to select relevant tools before calling main model.
<u>Tool retry</u>	Automatically retry failed tool calls with exponential backoff.
<u>LLM tool emulator</u>	Emulate tool execution using an LLM for testing purposes.
<u>Context editing</u>	Manage conversation context by trimming or clearing tool uses.
<u>Shell tool</u>	Expose a persistent shell session to agents for command execution.
<u>File search</u>	Provide Glob and Grep search tools over filesystem files.

中间件举例

- `PIIMiddleware` :在发送至模型前屏蔽敏感信息
- `SummarizationMiddleware` :对话历史过长时自动进行内容压缩
- `HumanInTheLoopMiddleware` :在工具调用执行之前，暂停代理执行，以便人工审批、编辑或拒绝工具调用。
- `ModelCallLimitMiddleware` ：限制模型调用次数，以防止无限循环或过高的成本。
- `ToolCallLimitMiddleware` ：通过限制工具调用次数来控制代理执行，可以全局限制所有工具的调用次数，也可以限制特定工具的调用次数。

这里演示一下PIIMiddleware

代码块

```
1  """
2  LangChain 1.0 中间件系统
3  展示PII检测
4  """
5
6  from langchain.agents import create_agent
```

```

7  from langchain.agents.middleware import (
8      PIIMiddleware
9  )
10 from langchain.tools import tool
11 from models import get_lc_model_client
12
13 # 定义缺失的工具函数
14 @tool
15 def send_email(recipient: str, subject: str, body: str) -> str:
16     """发送邮件的工具"""
17     return f"邮件已发送给 {recipient}, 主题: {subject}"
18
19 @tool
20 def read_user_data(user_id: str) -> str:
21     """读取用户数据的工具"""
22     return f"用户 {user_id} 的数据: 姓名张三, 邮箱 zhangsan@example.com, 手机号 13800138000"
23
24 def middleware_demo():
25     """中间件功能演示 - LangChain 1.0 独有特性"""
26
27     # 先初始化模型
28     model = get_lc_model_client()
29
30     # 创建带有多中间件的agent
31     agent = create_agent(
32         model=model,
33         tools=[send_email, read_user_data],
34         middleware=[
35             # PII检测和处理中间件
36             PIIMiddleware(
37                 "email",
38                 strategy="redact", # 遮掩策略
39                 apply_to_input=True,
40                 apply_to_output=True
41             ),
42             PIIMiddleware(
43                 "ip", # 使用 ip 类型作为手机号遮掩
44                 detector=r"(?:13[0-9]|14[01456879]|15[0-35-9]|16[2567]|17[0-8]|18[0-9]|19[0-35-9])\d{8}", # 手机号正则
45                 strategy="redact",
46                 apply_to_input=True,
47                 apply_to_output=True
48             ),
49             PIIMiddleware(
50                 "credit_card", # 使用 credit_card 作为身份证号遮掩
51                 detector=r"\d{17}[\dXx]", # 身份证号正则

```

```

52         strategy="redact",
53         apply_to_input=True,
54         apply_to_output=True
55     )
56 ],
57     system_prompt="你是一个安全的数据处理助手，严格保护用户隐私"
58 )
59
60 # 测试场景1: PII检测
61 print("\n--- 测试场景1: PII数据处理 ---")
62 try:
63     result = agent.invoke({
64         "messages": [{
65             "role": "user",
66             "content": "请读取用户001的数据"
67         }]
68     })
69
70     last_message = result["messages"][-1]
71     content = getattr(last_message, 'content', '') if
hasattr(last_message, 'content') else ''
72     print(f"PII处理后的输出: {content}")
73
74 except Exception as e:
75     print(f"PII测试错误: {e}")
76
77 # 测试场景2: 包含敏感信息的输入
78 print("\n--- 测试场景2: 敏感输入过滤 ---")
79 try:
80     result = agent.invoke({
81         "messages": [{
82             "role": "user",
83             "content": "我的手机号是13912345678，请发邮件给manager@company.com
告知我的信息"
84         }]
85     })
86
87     # 展示中间件处理结果
88     print("中间件处理结果:")
89     for i, msg in enumerate(result["messages"]):
90         msg_type = type(msg).__name__
91         content = str(getattr(msg, 'content', ''))[:150] if hasattr(msg,
'content') else ''
92         print(f" {i+1}. {msg_type}: {content}...")
93
94 except Exception as e:
95     print(f"敏感输入测试错误: {e}")

```

```
96
97     if __name__ == "__main__":
98         middleware_demo()
```

--- 测试场景1: PII数据处理 ---

PII处理后的输出: 用户001的数据如下:

```
- 姓名: 张三
- 邮箱: [REDACTED_EMAIL]
- 手机号: [REDACTED_IP]
```

--- 测试场景2: 敏感输入过滤 ---

中间件处理结果:

```
1. HumanMessage: 我的手机号是 [REDACTED_IP], 请发邮件给 manager@company.com 告知我的信息 ...
2. AIMessage: 我明白了, 但是为了保护您的隐私, 我不能在邮件中包含您的个人信息。如果您确定要发送邮件, 请确认。...
```

自定义中间件

通过在代理执行流程的特定点运行钩子来构建自定义中间件。

- 基于装饰器的: 适用于单钩中间件的快速简便

```
# Node-style: logging before model calls
@before_model
def log_before_model(state: AgentState, runtime: Runtime)
    print(f>About to call model with {len(state['messages'])} messages
    return None
```

- 基于类的: 对于具有多个钩子的复杂中间件来说功能更强大

```
class LoggingMiddleware(AgentMiddleware):
    def before_model(self, state: AgentState, runtime: Runtime)
        print(f>About to call model with {len(state['messages'])} messages
        return None

    def after_model(self, state: AgentState, runtime: Runtime)
        print(f"Model returned: {state['messages'][-1].content}
        return None
```

案例

我们可以自定义一个脱敏中间件。△以下代码仅为了演示如何创建自定义中间件, 本阶段无需掌握!

```

1  import os
2  import re
3
4  # 验证模型连接
5  from langchain_community.chat_models import ChatTongyi
6  from langchain_core.messages import HumanMessage
7
8  llm = ChatTongyi(api_key=os.environ.get("DASHSCOPE_API_KEY"))
9
10 # 创建自定义脱敏中间件
11 from langchain.agents.middleware import AgentMiddleware
12 from typing import Any, Dict
13 import re
14
15 class FixedSensitiveDataMiddleware(AgentMiddleware):
16     """修复的脱敏中间件 - 使用正确的参数签名"""
17
18     def __init__(self, patterns: list = None):
19         super().__init__()
20         self.patterns = patterns or [
21             (r'[a-zA-Z0-9_+-.]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-\.]+', '[EMAIL]'),
22             (r'(\+86)?1[3-9]\d{9}', '[PHONE]')
23         ]
24
25     def _desensitize_text(self, text: str) -> str:
26         # 如果内容为空或已经包含脱敏标记，则跳过处理
27         if not text or '[EMAIL]' in text or '[PHONE]' in text:
28             return text
29         # 快速预检查：只有当可能包含敏感信息时才继续处理
30         if '@' not in text and not re.search(r'1[3-9]\d{9}', text):
31             return text
32         print(f"脱敏前: {text}")
33         original_text = text
34         for pattern, replacement in self.patterns:
35             text = re.sub(pattern, replacement, text)
36         # 只有当内容发生变化时才打印
37         if original_text != text:
38             print(f"脱敏后: {text}")
39         return text
40
41     def before_model(self, state: Dict[str, Any]) -> Dict[str, Any]:
42         """在模型调用前处理 - 修复参数签名"""
43         if 'messages' in state:
44             messages = state['messages']
45             processed_any = False
46             for message in messages:

```



```

47         if hasattr(message, 'content') and isinstance(message.content,
48 str):
49             # 只处理非空内容且未被脱敏的内容
50             if message.content and '[EMAIL]' not in message.content and
51 '[PHONE]' not in message.content:
52                 # 快速预检查: 只有当可能包含敏感信息时才继续处理
53                 if '@' in message.content or re.search(r'1[3-9]\d{9}',
54 message.content):
55                     # 只有在真正需要处理时才打印日志
56                     if not processed_any:
57                         print("中间件 before_model 被调用")
58                         original_content = message.content
59                         message.content =
60 self._desensitize_text(message.content)
61                     # 只有当内容发生变化时才打印
62                     if original_content != message.content:
63                         print(f"处理消息内容: {original_content}")
64                         print(f"消息内容已从 '{original_content}' 修改为
65 '{message.content}'")
66                     processed_any = True
67                 if processed_any:
68                     print("中间件 after_model 被调用")
69                 return state
70
71     def after_model(self, state: Dict[str, Any]) -> Dict[str, Any]:
72         """在模型调用后处理 - 修复参数签名"""
73         # 只有在真正处理了内容时才打印
74         # 这里可以添加后处理逻辑
75         return state
76
77 # 创建修复后的中间件
78 fixed_middleware = FixedSensitiveDataMiddleware()
79
80 # ===== 测试中间件功能 =====
81
82 print("\n" + "=" * 50)
83 print("开始测试中间件功能")
84 print("=" * 50)
85
86 try:
87     # 导入必要的组件
88     from langchain_community.agent_toolkits.load_tools import load_tools
89     from langgraph.checkpoint.memory import InMemorySaver
90     from langchain.agents import create_agent
91
92     # 导入工具
93     tools = load_tools(["arxiv"])

```

```

89
90     # 创建短期记忆实例
91     memory = InMemorySaver()
92
93     # 系统提示词设计
94     system_prompt = "你是一个专业的论文查询助手，使用arxiv工具为用户查询论文信息，回答
需简洁准确，包含论文标题、作者、发表时间和核心摘要。"
95
96     # 创建带中间件的Agent
97     agent_with_middleware = create_agent(
98         model=llm,
99         tools=tools,
100        system_prompt=system_prompt,
101        checkpointer=memory,
102        middleware=[fixed_middleware]
103    )
104    print("中间件版本创建成功！")
105
106    # 测试---
107    # 测试：包含电子邮件的输入
108    print("测试：电子邮件脱敏")
109    email_input = "我的邮箱是test.user@example.com，请帮我查询论文1605.08386"
110    print(f"输入内容：{email_input}")
111
112    result1 = agent_with_middleware.invoke(
113        {"messages": [{"role": "user", "content": email_input}]},
114        config={"configurable": {"thread_id": "middleware_test_1"}}
115    )
116    print("结果:", result1["messages"][-1].content)
117
118    except Exception as e:
119        print(f"中间件版本仍然失败：{e}")

```

=====

开始测试中间件功能

=====

中间件版本创建成功！
 测试：电子邮件脱敏
 输入内容：我的邮箱是test.user@example.com，请帮我查询论文1605.08386
 中间件 before_model 被调用
 脱敏前：我的邮箱是test.user@example.com，请帮我查询论文1605.08386
 脱敏后：我的邮箱是[EMAIL]，请帮我查询论文1605.08386
 处理消息内容：我的邮箱是test.user@example.com，请帮我查询论文1605.08386
 消息内容已从 '我的邮箱是test.user@example.com，请帮我查询论文1605.08386' 修改为 '我的邮箱是[EMAIL]，请帮我查询论文1605.08386'
 中间件 after_model 被调用
 结果：论文信息如下：

标题：Heat-bath random walks with Markov bases
 作者：Caprice Stanley, Tobias Windisch
 发表时间：2016-05-26

摘要：研究了由有限允许移动集生成的格点图，这些图的边可以是任意长度的移动。我们证明了在固定整数矩阵的纤维上，这些图的直径可以从上面被一个常数限制。然后研究了这些图上的热浴随机游走的混合行为。我们还给出了移动集的显式条件，使得热浴随机游走（Glauber动力学的一种推广）在固定维度下是一个扩展器。